SC2001 Algorithm Design & Analysis
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-23 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

Chapter 1

Foundations: From Zero to Algorithmic Reasoning

> *"An algorithm must be seen to be believed."* — Donald Knuth. Before we design faster algorithms, we must agree on what an algorithm *is*, how to argue that it is *correct*, and what it costs to run. This chapter builds that floor from first principles.

Prerequisites: None beyond elementary logic and high-school algebra, but we assume the maturity of MH1812 Discrete Mathematics and SC1007 Data Structures. Students who have not seen sets, functions, or proof by induction will find a self-contained refresher here. Readers already comfortable with induction may skim §1.3 and focus on §1.5–1.6.

Learning Outcomes

After this chapter you will be able to:

(L1) recall essential discrete-mathematical notation and handle sums, logarithms, floors and ceilings fluently;

(L2) distinguish and apply direct proof, contrapositive, contradiction, and counterexamples;

(L3) prove statements by (weak and strong) mathematical induction;

(L4) define *algorithm* precisely and describe the RAM model and its cost measure;

(L5) prove correctness of iterative algorithms using loop invariants and termination arguments.

1.1 Discrete Mathematics Refresher

We collect notation and facts used without comment throughout the book. This is not a substitute for MH1812, but a compact reference.

1.1.1 Sets, Functions, Relations

A *set* is an unordered collection of distinct objects. We write $x \in S$, $S \subseteq T$, $S \subsetneq T$, $|S|$ for cardinality, $\emptyset$ for the empty set, $\mathbb{N} = \{0, 1, 2, \dots\}$ (we include 0; any off-by-one is irrelevant asymptotically), $\mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{Z}^+$.

A *function* $f : A \to B$ assigns to each $a \in A$ exactly one $b = f(a) \in B$. A is the domain, B the codomain, $f(A) = \{f(a) : a \in A\}$ the image. f is *injective* if $f(a) = f(a') \Rightarrow a = a'$, *surjective* if $f(A) = B$, *bijective* if both. The *composition* $(g \circ f)(a) = g(f(a))$.

A *relation* $R \subseteq A \times A$ may be reflexive, symmetric, transitive, antisymmetric — yielding equivalences and partial orders. A *graph* $G = (V, E)$ with $E \subseteq V \times V$ appears from Chapter 6; we assume familiarity with paths, cycles, and trees from SC1007.

1.1.2 Logic and Quantifiers

Propositions combine via $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$. The quantifiers $\forall$ (*for all*) and $\exists$ (*there exists*) bind variables: $\forall n \in \mathbb{N},\ P(n)$ and $\exists x \in S,\ Q(x)$. Negation swaps them:

$$\neg(\forall x\, P(x)) \equiv \exists x\, \neg P(x), \qquad \neg(\exists x\, P(x)) \equiv \forall x\, \neg P(x).$$

Implication $P \Rightarrow Q$ is equivalent to $\neg P \vee Q$ and to its contrapositive $\neg Q \Rightarrow \neg P$.

1.1.3 Sums, Products, Logarithms

We use freely:

- Arithmetic sum: $\sum_{i=1}^{n} i = n(n+1)/2$.
- Geometric sum ($r \neq 1$): $\sum_{i=0}^{n-1} r^i = (r^n - 1)/(r - 1)$; if $|r| < 1$, $\sum_{i=0}^{\infty} r^i = 1/(1-r)$.
- Harmonic sum: $H_n = \sum_{i=1}^{n} 1/i = \ln n + \gamma + O(1/n)$ where $\gamma \approx 0.5772$.
- Logarithms: log without base means $\log_2$ in this book when analysing algorithms; ln is base e, lg is always base 2. Change of base: $\log_a n = (\log_b n)/(\log_b a) = \Theta(\log_b n)$. Hence base is irrelevant inside $O(\cdot)$.
- Floors and ceilings: $\lfloor x \rfloor \leq x < \lfloor x \rfloor + 1$, $\lceil x \rceil - 1 < x \leq \lceil x \rceil$, and $x - 1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x + 1$. We often drop $\lfloor \cdot \rfloor, \lceil \cdot \rceil$ in asymptotics, noting the error is at most 1.
- Factorials and exponentiation, inequalities: $n! \leq n^n$, $2^n \leq n!$ for $n \geq 4$, $(1 + 1/n)^n < e < (1 + 1/n)^{n+1}$.

Example 1.1 (Manipulating sums)**.** For $n \geq 1$,

$$\sum_{i=1}^{n} (2i - 1) = 2\frac{n(n+1)}{2} - n = n^2.$$

Linearity of summation ($\sum c \cdot a_i = c \sum a_i$, $\sum (a_i + b_i) = \sum a_i + \sum b_i$) justifies the first equality without induction.

1.2 Proof Techniques

Every correctness proof and every lower bound in this book uses one of four patterns. Master them now.

Definition 1.1 (Theorem, Lemma, Corollary)**.** A *theorem* is a statement proved true. A *lemma* is a helper theorem en route to a larger one. A *corollary* is an immediate consequence.

1.2.1 Direct Proof

Assume the hypothesis; deduce the conclusion via definitions and previously proved facts.

Example 1.2. Claim: If n is odd then n^2 is odd. *Proof.* Write $n = 2k + 1$ for some $k \in \mathbb{Z}$. Then $n^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$, which is odd. $\square$

1.2.2 Contrapositive

To prove $P \Rightarrow Q$, prove $\neg Q \Rightarrow \neg P$ instead (logically equivalent).

Example 1.3. Claim: If n^2 is even then n is even. *Proof.* Contrapositive: if n is odd then n^2 is odd — proved above. □

1.2.3 Proof by Contradiction

Assume the statement is false and derive an impossibility ($R \wedge \neg R$).

Example 1.4 (Irrationality of $\sqrt{2}$)**.** Suppose $\sqrt{2} = p/q$ in lowest terms, $p, q \in \mathbb{Z}^+$, $\gcd(p, q) = 1$. Then $2q^2 = p^2$, so p^2 is even, hence p is even: $p = 2k$. Then $2q^2 = 4k^2$, so $q^2 = 2k^2$, hence q is even — contradicting $\gcd(p, q) = 1$. □

1.2.4 Counterexample and Cases

To disprove $\forall x\, P(x)$ it suffices to exhibit one x with $\neg P(x)$. To prove a claim that naturally splits (e.g., by parity, by size), prove each case separately and exhaust all possibilities.

Example 1.5. Claim "Every integer $n > 1$ is prime" is false: $4 = 2 \cdot 2$ is a counterexample. Claim "$n^2 \bmod 4 \in \{0, 1\}$ for all $n \in \mathbb{Z}$": check cases n even ($n = 2k \Rightarrow n^2 = 4k^2 \equiv 0$) and n odd ($n = 2k + 1 \Rightarrow n^2 = 4k(k + 1) + 1 \equiv 1$).

Remark 1.1 (How to choose). Try direct proof first. If the conclusion is a negation ("not", "irrational", "no algorithm exists"), contradiction is often cleaner. If P is itself a negation, contrapositive may be direct in disguise.

1.3 Mathematical Induction

Induction is the single most used proof technique in algorithm analysis: it proves loop invariants, closed forms for recurrences, and properties of recursively defined structures.

1.3.1 Weak (Ordinary) Induction

To prove $\forall n \geq n_0$, $P(n)$:

(I1) **Base case:** prove $P(n_0)$.

(I2) **Inductive step:** prove $\forall k \geq n_0$, $P(k) \Rightarrow P(k + 1)$.

The hypothesis $P(k)$ in the inductive step is the *induction hypothesis* (IH).

Theorem 1.1 (Sum of first n integers)**.** For all $n \geq 1$, $\sum_{i=1}^{n} i = n(n + 1)/2$.

Proof. Base $n = 1$: $1 = 1 \cdot 2/2$. Assume IH for k: $\sum_{i=1}^{k} i = k(k + 1)/2$. Then

$$\sum_{i=1}^{k+1} i = \Big(\sum_{i=1}^{k} i\Big) + (k + 1) = k(k + 1)/2 + (k + 1) = (k + 1)(k + 2)/2.$$

Thus $P(k) \Rightarrow P(k + 1)$. By induction $P(n)$ holds for all $n \geq 1$. □

1.3.2 Strong Induction

To prove $\forall n \geq n_0$, $P(n)$, prove $P(n_0)$ and $\forall k \geq n_0$, $(\forall j,\, n_0 \leq j \leq k \Rightarrow P(j)) \Rightarrow P(k+1)$. The IH assumes *all* earlier cases, not just k. Strong and weak induction are logically equivalent, but strong induction avoids reproving ad hoc "reach back" lemmas.

Theorem 1.2 (Every $n \geq 2$ has a prime divisor)**.** For all $n \geq 2$, n is divisible by some prime.

Proof. Base $n = 2$: 2 is prime and divides itself. Fix $k \geq 2$ and assume every integer j with $2 \leq j \leq k$ has a prime divisor (strong IH). Consider $k+1$: if $k+1$ is prime we are done; otherwise $k + 1 = a \cdot b$ with $2 \leq a, b \leq k$. By IH, a has a prime divisor p, and $p \mid a \mid (k+1)$. □

Theorem 1.3 (Fibonacci growth — strong induction at work)**.** Let $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$. Then $F_n \geq 2^{n/2-1}$ for all $n \geq 1$ (a weak lower bound used later to analyse recursive Fibonacci).

Proof. Check $n = 1$: $F_1 = 1 \geq 2^{-0.5} \approx 0.707$. $n = 2$: $F_2 = 1 \geq 2^0 = 1$ — equality, but we need two bases because the recurrence reaches back two steps. Assume IH for all $1 \leq j \leq k$ with $k \geq 2$. Then $F_{k+1} = F_k + F_{k-1} \geq 2^{k/2-1} + 2^{(k-1)/2-1} = 2^{(k-1)/2-1}(2^{1/2}+1) \geq 2^{(k-1)/2-1} \cdot 2 = 2^{(k+1)/2-1}$, using $\sqrt{2} + 1 > 2$. □

1.3.3 Structural Induction

For recursively defined sets (lists, trees, well-formed formulae), prove P for base structures and that constructors preserve P. This is induction on the *structure*, not merely on size, and underpins correctness of recursive algorithms on trees/graphs in Chapters 5–6.

Example 1.6 (Binary trees)**.** A binary tree is either Leaf or $\mathsf{Node}(L, x, R)$ where L, R are binary trees. To prove "number of leaves = number of internal nodes +1" for full binary trees, check the leaf base and show the constructor preserves the invariant.

Remark 1.2 (Common pitfalls)*.* Always *state* the IH explicitly, check that the base covers all cases the step reaches back to, and avoid "all horses are the same colour"—style illicit generalisation where $P(k) \Rightarrow P(k+1)$ fails for small k.

1.4 What Is an Algorithm?

Definition 1.2 (Algorithm, informal)**.** An *algorithm* is a finite, unambiguous sequence of elementary steps that, given any admissible input, terminates after finitely many steps and produces the required output.

Knuth and later theory make this precise: an algorithm satisfies five criteria — *finiteness* (description is finite), *definiteness* (each step is precisely defined), *input* (zero or more inputs from a specified set), *output* (one or more outputs with a specified relation to the inputs), *effectiveness* (steps are basic enough to be carried out, in principle, by hand) — plus, crucially, *termination on every admissible input.* A procedure that may run forever is a *computational method*, not an algorithm.

An *algorithm* is distinct from a *program*: the former is an abstract idea, the latter its realisation in a programming language on concrete hardware. The same algorithm (e.g., binary search) can be implemented in Python, Java, or C with different constant-factor performance but the same asymptotic behaviour — which is why we analyse the idea, not the code.

1.4.1 Pseudocode Conventions in This Book

We write algorithms in language-agnostic pseudocode, one-indexed for arrays (as in CLRS), with `//` comments, assignment $\leftarrow$, comparison $=, \neq, <, \leq$, and control flow `if-then-else`, `while`, `for`, `return`. Indentation denotes block structure. Procedures are named in SmallCaps. We count *primitive operations* (defined in §1.5) rather than wall-clock time.

Example 1.7 (Euclid's algorithm — the oldest non-trivial algorithm).

```
EUCLID(a, b)    // assume a ≥ b ≥ 0, integers

1  while b ≠ 0 do
2      r ← a mod b
3      a ← b
4      b ← r
5  return a
```

We will prove in §1.6 that this returns $\gcd(a, b)$ and terminates, and in Chapter 2 that it runs in $O(\log \min(a, b))$ divisions — exponentially faster than naive search.

1.5 The Random Access Machine (RAM) Model

To compare algorithms fairly we need a shared model of "how long a step takes". The workhorse is the *Random Access Machine*, an idealised von Neumann computer that abstracts away caches, pipelines, and operating-system noise while preserving the essence of modern hardware.

1.5.1 Components

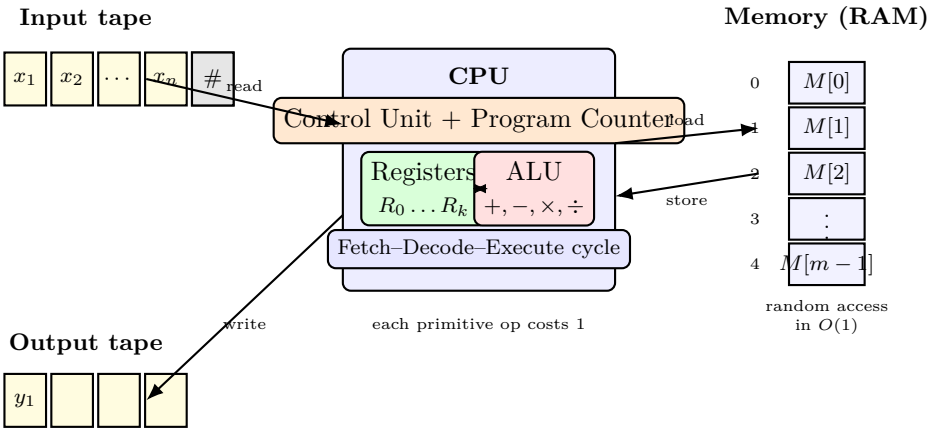


Figure 1.1: The Random Access Machine (RAM). The CPU fetches instructions pointed to by the program counter, operates on registers and the ALU, and accesses any memory cell $M[i]$ in unit time. Input is read sequentially; output is write-only. This idealisation underlies all running-time analysis in the book.

Figure 1.1 summarises the model. Concretely:

- **Memory:** an array $M[0 \ldots m-1]$ of *words* (integers), each large enough to hold any input value or pointer ($\Theta(\log n)$ bits). Any cell is accessed in $O(1)$ time — the "random access".
- **CPU:** a finite set of registers, an arithmetic-logic unit (ALU), and a program counter pointing to the next instruction.
- **Instruction set:** `LOAD`, `STORE`, `ADD`, `SUB`, `MUL`, `DIV`, `MOD`, `COMPARE`, `JUMP`, `JZ/JNZ`, `READ`, `WRITE`, etc. Each *primitive* operation costs one unit of time (the *uniform-cost* criterion).

- **Program:** a finite list of instructions stored in a separate program memory; it does not count toward input size.

1.5.2 Cost Model and the Word-RAM Refinement

Under the uniform-cost RAM, the *running time* of an algorithm on an input of size n is the number of primitive steps executed. Input size is measured in words (or bits when bit-complexity matters). For most of this book uniform cost is adequate because numbers fit in a word.

When numbers grow huge (cryptography, arbitrary-precision arithmetic), we switch to the *logarithmic-cost* or *word-RAM* model: a word holds $w = \Theta(\log n)$ bits, arithmetic on w-bit words costs $O(1)$, but operating on b-bit integers costs $\Theta(b/w)$. We flag explicitly when bit-complexity matters; otherwise assume uniform cost.

1.5.3 Why This Model?

The RAM abstracts away real hardware — no cache hierarchy, no branch prediction, no parallelism — yet it predicts relative performance remarkably well: an $O(n \log n)$ RAM algorithm beats an $O(n^2)$ one on real machines once n exceeds a modest threshold, regardless of constant factors. It also lets us reason *before* implementing. Its limitations (ignoring memory hierarchy and parallelism) are addressed in advanced courses; for SC2001 it is the agreed accounting rule.

Example 1.8 (Cost of linear search on RAM)**.** LINEARSEARCH(A, n, key)

```
1  for i ← 1 to n do
2      if A[i] = key then return i
3  return NIL
```

Each loop iteration performs $O(1)$ RAM operations (one array access, one comparison, one branch). Hence the running time is proportional to the number of iterations — the quantity we formalise in Chapter 2 as $O(n)$ worst-case.

1.6 Correctness: Loop Invariants and Termination

An algorithm is useless if it returns the wrong answer or never stops. *Correctness* has two parts:

- **Partial correctness:** *if* the algorithm terminates, its output satisfies the specification.
- **Termination:** the algorithm terminates on every admissible input.

Together they give *total correctness*. For iterative algorithms, partial correctness is proved via *loop invariants*; termination via a *variant* (a quantity that strictly decreases and is bounded below).

Definition 1.3 (Loop invariant)**.** A predicate I on the program state is a *loop invariant* for a loop with guard G if:

(LI1) **Initialisation:** I holds before the first iteration.

(LI2) **Maintenance:** if I holds before an iteration and G is true, then I holds after that iteration.

(LI3) **Termination:** when the loop exits ($\neg G$), $I \wedge \neg G$ implies the desired postcondition.

Think of I as the "inductive hypothesis travelling around the loop" — its proof *is* an induction on the number of iterations.

We also need a *variant* v — an integer expression such that each iteration strictly decreases v and v is bounded below (usually $v \geq 0$). Existence of a variant guarantees the loop cannot run forever.

Example 1.9 (Insertion sort — full correctness proof). INSERTIONSORT$(A[1 \dots n])$

```
1  for j ← 2 to n do
2      key ← A[j]
3      i ← j − 1
4      while i > 0 and A[i] > key do
5          A[i + 1] ← A[i]
6          i ← i − 1
7      A[i + 1] ← key
```

Specification: upon termination, $A[1 \dots n]$ is a permutation of the input and is non-decreasing.

Outer-loop invariant: At the start of each iteration j $(2 \le j \le n+1)$, $A[1 \dots j-1]$ consists of the original elements $A_0[1 \dots j-1]$ in sorted order, and $A[j \dots n]$ is untouched.

Initialisation $(j = 2)$: $A[1 \dots 1]$ is trivially sorted. *Maintenance:* Assume invariant at j. The inner while shifts the sorted block $A[1 \dots j-1]$ right until the correct position for $key = A_0[j]$ is found, then inserts key. Thus $A[1 \dots j]$ is sorted and contains exactly $A_0[1 \dots j]$. So invariant holds for $j+1$. *Termination:* Loop exits when $j = n+1$; invariant gives $A[1 \dots n]$ sorted and a permutation of $A_0[1 \dots n]$.

Inner loop: Invariant: $A[i+2 \dots j]$ (if $i < j-1$) contains the original $A[1 \dots j-1]$ elements greater than key, shifted by one, and $A[1 \dots i]$ remains sorted and $\le key$ candidates. Variant: i, strictly decreasing and bounded below by 0 — so inner loop terminates. Outer loop variant is $n - j$, also decreasing. Hence total correctness.

Example 1.10 (Euclid terminates)**.** Variant $v = b$ (the second argument). Each iteration replaces (a, b) by $(b, a \bmod b)$ with $0 \le a \bmod b < b$, so v strictly decreases and stays ≥ 0. Hence the while loop must exit. Combined with invariant "$\gcd(a, b)$ is unchanged" (since $\gcd(a, b) = \gcd(b, a \bmod b)$), the returned a equals the original gcd.

Remark 1.3 (Recipe for students). When you write a loop invariant, answer three questions: What has been done so far? What remains? Where is the boundary? The invariant should mention the boundary index and state precisely what is true on each side. Vague invariants ("array is partially sorted") are not invariants at all.

1.7 Chapter Notes

The RAM model is due to Cook and Reckhow (1973) and formalised by Aho, Hopcroft, and Ullman. Our uniform-cost version follows CLRS Ch. 2; the word-RAM refinement is standard in modern algorithm theory (see Demaine, "Cache-Oblivious Algorithms"). Loop invariants are Hoare logic in miniature (C. A. R. Hoare, 1969); the full axiomatic method is covered in SC3270 Reasoning About Programs. For discrete mathematics, Rosen's *Discrete Mathematics and Its Applications* and the MH1812 lecture notes remain the definitive references.

1.8 Exercises

E1.1 **Warm-up: logic and quantifiers.** Formalise "Every SC2001 student who passed MH1812 can follow an induction proof" using predicates, then write its negation without using $\neg$ directly in front of a quantifier. Explain why the negation is *not* "No SC2001 student who passed MH1812 can follow an induction proof."

E1.2 **Induction — weak vs. strong.** Prove by induction that for all $n \ge 1$,

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}.$$

Then prove that every integer $n \geq 12$ can be written as $4a + 5b$ for non-negative integers a, b. Which proof needed strong induction, and why? (Hint for the second: check bases $12, 13, 14, 15$ and step by 4.)

E1.3 **RAM cost model.** Consider:

```
SumArray(A[1...n])
1  s ← 0
2  for i ← 1 to n do s ← s + A[i]
3  return s
```

(a) Count RAM primitive operations per line and give the total running time as a function of n (up to constant factors). (b) Suppose each $A[i]$ is a b-bit integer with $b \gg \log n$. How does the answer change under the word-RAM model with word size $w = \Theta(\log n)$? (c) Would you call this algorithm $O(n)$ or $O(nb/w)$? Justify.

E1.4 **Loop invariant — find the bug.** A student proposes this invariant for LinearSearch: "At the start of iteration i, *key* is not in $A[1 \dots i-1]$." Prove that if the loop invariant method is applied correctly, LinearSearch is correct. Then give a buggy variant of the algorithm where the same invariant still holds on every iteration but the algorithm is *incorrect* upon termination, and explain which of (LI1)–(LI3) fails.

E1.5 **Euclid's invariant in full.** Prove the invariant "$\gcd(a, b)$ is preserved" for Euclid's algorithm from §1.4: show (a) $\gcd(a, b) = \gcd(b, a \bmod b)$ for $b > 0$, (b) the invariant holds initially, is maintained, and implies the postcondition when $b = 0$, and (c) the number of iterations is $O(\log b)$ by showing $a \bmod b < a/2$ when $b \leq a$. This foreshadows the logarithmic analysis in Chapter 2.

Chapter 2

Asymptotic Analysis

> *"We should forget about small efficiencies, say about 97% of the time."* — Knuth. Asymptotic notation lets us forget the 97% and compare the shape of growth itself.

Prerequisites: Ch. 1 (RAM model, induction, sums/logs). This chapter formalises the $O(\cdot)$ language used in every later analysis.

Learning Outcomes

After this chapter you will be able to:

(L1) state the five asymptotic notations with quantifiers and negate them correctly;

(L2) apply the limit method to classify functions;

(L3) order the hierarchy $\log n \prec n \prec n \log n \prec n^2 \prec 2^n$ and justify it;

(L4) distinguish best-, worst-, and average-case analysis;

(L5) write a recurrence for a recursive algorithm's running time (bridge to Ch. 3).

2.1 Why Asymptotics?

RAM time is $T(n) = \sum(\text{cost}_i \times \text{count}_i(n))$, an unwieldy exact formula. Constants depend on compiler and hardware; what survives across machines is *rate of growth*. Asymptotic notation suppresses constants and lower-order terms and compares $T(n)$ to a clean reference $g(n)$ for large n only.

We consider functions $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$ eventually non-negative. "Large n" is captured by $\exists n_0 \, \forall n \geq n_0$.

2.2 The Five Notations

Definition 2.1 (O — asymptotic upper bound)**.** $f(n) = O(g(n))$ iff $\exists c > 0 \, \exists n_0 \in \mathbb{N} \, \forall n \geq n_0 : 0 \leq f(n) \leq c\, g(n)$.

Definition 2.2 (Ω — asymptotic lower bound)**.** $f(n) = \Omega(g(n))$ iff $\exists c > 0 \, \exists n_0 \, \forall n \geq n_0 : \; 0 \leq c\, g(n) \leq f(n)$.

Definition 2.3 (Θ — tight bound)**.** $f(n) = \Theta(g(n))$ iff $\exists c_1, c_2 > 0 \, \exists n_0 \, \forall n \geq n_0 : \; 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$. Equivalently $f = O(g)$ and $f = \Omega(g)$.

Definition 2.4 (o — strict upper bound)**.** $f(n) = o(g(n))$ iff $\forall c > 0\, \exists n_0\, \forall n \geq n_0 :\ 0 \leq f(n) < c\, g(n)$. That is, $f/g \to 0$.

Definition 2.5 (ω — strict lower bound)**.** $f(n) = \omega(g(n))$ iff $\forall c > 0\, \exists n_0\, \forall n \geq n_0 :\ 0 \leq c\, g(n) < f(n)$. That is, $f/g \to \infty$.

Remark 2.1 (Quantifier order matters). O has $\exists c\, \exists n_0\, \forall n \geq n_0$; o has $\forall c\, \exists n_0\, \forall n \geq n_0$. Negations swap quantifiers: $f \neq O(g)$ iff $\forall c > 0\, \forall n_0\, \exists n \geq n_0 : f(n) > c\, g(n)$. The abuse $f = O(g)$ means $f \in O(g)$ as a set.

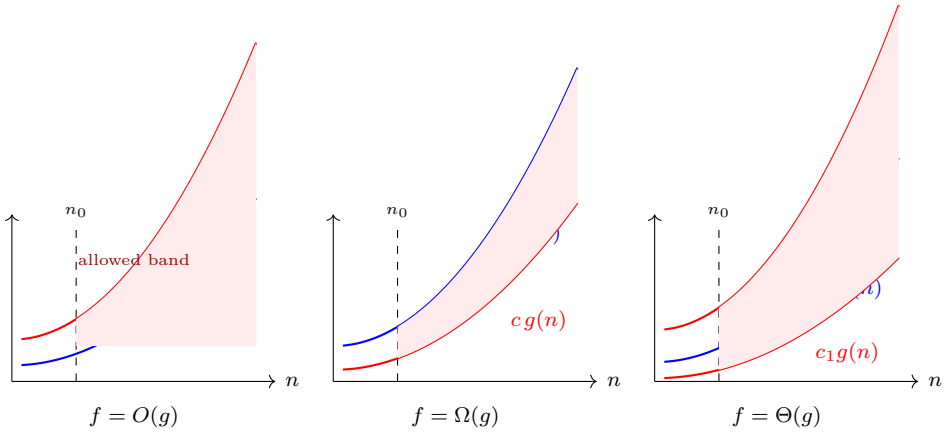


Figure 2.1: O, Ω, and Θ as eventually-sandwiching bands. Only behaviour for $n \geq n_0$ matters; constants c, c_1, c_2 scale the reference $g(n)$.

Fig. 2.1 visualises the three bands. o and ω require the sandwich to hold for *every* $c > 0$ (arbitrarily tight).

2.3 The Limit Method

When limits exist, quantifier gymnastics reduces to calculus.

Theorem 2.1 (Limit test)**.** Let $L = \lim_{n\to\infty} f(n)/g(n)$ (in $\mathbb{R}_{\geq 0} \cup \{\infty\}$), assuming $g(n) > 0$ eventually.

(a) If $0 < L < \infty$ then $f = \Theta(g)$.

(b) If $L = 0$ then $f = o(g)$ (hence $f = O(g)$).

(c) If $L = \infty$ then $f = \omega(g)$ (hence $f = \Omega(g)$).

If L does not exist (oscillation), the test is inconclusive and we return to definitions.

Example 2.1. $3n^2 + 5n + 20 = \Theta(n^2)$ because $(3n^2 + 5n + 20)/n^2 \to 3 \in (0, \infty)$. Similarly $\log(n!) = \Theta(n \log n)$ since $n! \leq n^n$ and $n! \geq (n/2)^{n/2}$, or via Stirling; we prove it rigorously in Ch. 3.

Example 2.2 (Limits need care)**.** $f(n) = n^{1+\sin n}$ oscillates between n^0 and n^2; $f(n)/n$ has no limit and f is neither $O(n)$ nor $\Omega(n^2)$. Always check existence.

Useful rules: l'Hôpital for ∞/∞, and $\log_a n = \Theta(\log_b n)$ for any $a, b > 1$ (change of base is a constant factor). Hence we write simply $\Theta(\log n)$.

2.4 Growth Hierarchy

Theorem 2.2 (Hierarchy)**.** For any $\epsilon > 0$ and $c > 1$:

$$1 \prec \log n \prec n^{\epsilon} \prec n \prec n \log n \prec n^{1+\epsilon} \prec n^2 \prec 2^n \prec n! \prec n^n,$$

where $a(n) \prec b(n)$ means $a(n) = o(b(n))$. In particular $\log n = o(n^{\epsilon})$, $n^k = o(c^n)$ for any fixed k, and $c^n = o(n!)$.

Sketch. Each claim is a limit: e.g. $\log n / n^{\epsilon} \to 0$ by l'Hôpital, and $n^k / c^n \to 0$ because exponentials dominate polynomials (apply l'Hôpital k times). The rest is transitivity of o. □

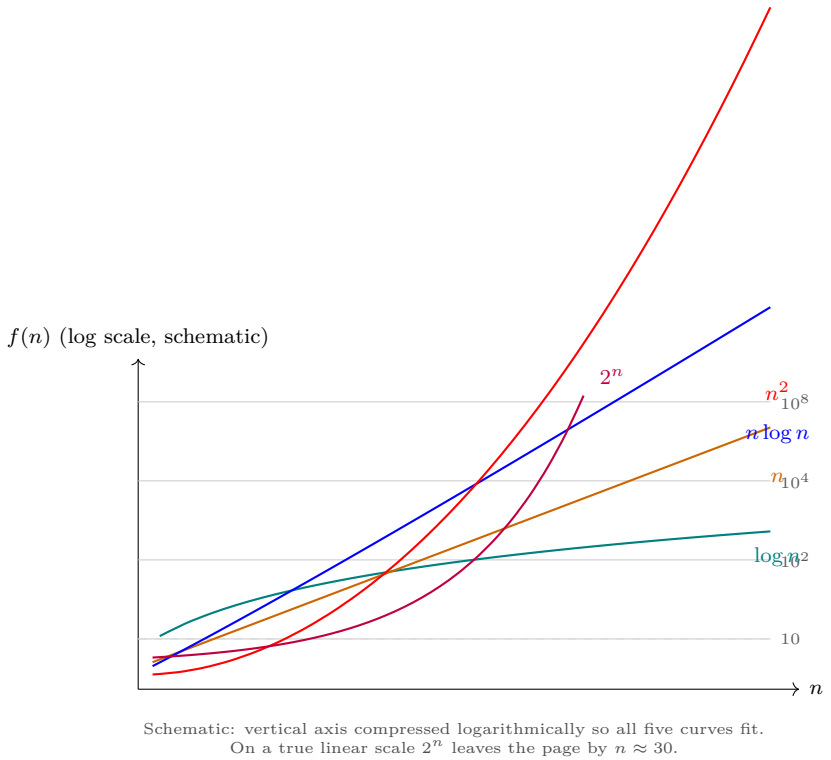


Figure 2.2: Growth hierarchy on a logarithmically compressed vertical axis. Steeper eventually dominates: $\log n \ll n \ll n \log n \ll n^2 \ll 2^n$.

Fig. 2.2 shows the separation. Consequence: an $O(n \log n)$ sorting algorithm will beat an $O(n^2)$ one for large n no matter the constants — the motivation for Ch. 4.

2.5 Best, Worst, and Average Case

For input size n, let $T(I)$ be RAM steps on instance I with $|I| = n$:

- **Worst case:** $T_{\text{worst}}(n) = \max_{|I|=n} T(I)$ — the guarantee.
- **Best case:** $T_{\text{best}}(n) = \min_{|I|=n} T(I)$ — rarely used alone (overly optimistic).
- **Average case:** $T_{\text{avg}}(n) = \mathbb{E}_{I \sim D_n}[T(I)]$ under a distribution D_n over size-n inputs (often uniform). Requires specifying D_n.

When we say "LinearSearch is $O(n)$" we mean $T_{\text{worst}}(n) = O(n)$; its best case is $\Theta(1)$ and its average case (uniform, key present) is $\Theta(n)$. InsertionSort is $\Theta(n)$ best case (already sorted, inner loop never runs) but $\Theta(n^2)$ worst case.

Average-case analysis needs randomness: we fix D_n, then analyse $\mathbb{E}[T]$. Randomised algorithms (Ch. 8) instead randomise the *algorithm* and bound $\mathbb{E}[T(I)]$ for every I.

2.6 Bridge to Recurrences

Recursive algorithms do not have closed-form $T(n)$ directly; they satisfy *recurrences*.

Example 2.3 (Merge sort preview)**.** MergeSort on n elements splits into two halves, recurses, then merges in $\Theta(n)$:

$$T(n) = 2\,T(n/2) + \Theta(n), \qquad T(1) = \Theta(1).$$

Solving gives $T(n) = \Theta(n \log n)$ — the canonical divide-and-conquer recurrence solved in Ch. 3.

In general a recurrence has form $T(n) = a\,T(n/b) + f(n)$ or $T(n) = T(n-1) + f(n)$ with base $T(n_0) = c_0$. Ch. 3 develops three systematic solvers: substitution (guess-and-verify by induction), recursion trees (summing level costs), and the Master Theorem for $aT(n/b) + f(n)$. Asymptotics from this chapter is the language in which those solutions are stated.

2.7 Chapter Notes

CLRS Ch. 3 is the standard reference for $O/\Theta/o/\omega$; Knuth (1976) introduced O, Ω, Θ notation into CS. The limit method is an application of real analysis — Rudin Ch. 3. Hierarchy facts follow from $\lim_{n\to\infty}(\log n)/n^{\epsilon} = 0$, proved via l'Hôpital. Average-case analysis connects to probability (MH1813); we revisit it for QuickSort and hashing.

2.8 Exercises

E2.1 **Definitions and negations.** Let $f(n) = 5n^2 + 3n$ and $g(n) = n^2$. (a) Prove $f = O(g)$ by exhibiting c, n_0. (b) Prove $f = \Omega(g)$. Conclude $f = \Theta(g)$. (c) Write the formal negation of "$h(n) = o(g(n))$" with quantifiers and use it to show $n^2 \neq o(n^2)$.

E2.2 **Limit method vs. oscillation.** Classify each pair using the limit method when applicable; otherwise return to definitions: (a) $f(n) = n \log n$, $g(n) = n^{1.5}$; (b) $f(n) = 2^n$, $g(n) = 3^n$; (c) $f(n) = n^{1+\sin n}$, $g(n) = n$. Determine which of $O, \Omega, \Theta, o, \omega$ hold (list all that apply).

E2.3 **Cases and distributions.** Consider LinearSearch for a key in $A[1 \ldots n]$ (distinct elements, key present uniformly at random position). (a) Give $T_{\text{best}}(n)$, $T_{\text{worst}}(n)$ with Θ bounds. (b) Compute $T_{\text{avg}}(n)$ exactly as a sum and give its Θ class. (c) Suppose the key is absent with probability $1/2$ and otherwise uniform among positions. How does $T_{\text{avg}}(n)$ change?

E2.4 **Setting up recurrences.** Write recurrences (with base cases) for: (a) RecursiveMax that finds the maximum by comparing $A[n]$ with the max of $A[1 \ldots n-1]$; (b) binary search on a sorted array of size n (worst case); (c) Karatsuba-style: $T(n) = 3T(n/2) + \Theta(n)$ — do not solve, just explain in one sentence what 3, $n/2$, and $\Theta(n)$ represent. Identify which cases from (a)–(c) fit the Master form $aT(n/b) + f(n)$.

Chapter 3

Recurrences

Recurrences describe the running time of recursive algorithms. Solving them—turning a recursive definition into a closed asymptotic bound—is the central analytic step in divide-and-conquer analysis. This chapter develops three complementary tools, highlights their limits, and applies them to MergeSort and Karatsuba multiplication.

3.1 From Algorithms to Recurrences

A typical divide-and-conquer algorithm on input size n divides into a subproblems of size n/b, then spends $f(n)$ time dividing and combining. Its worst-case time satisfies

$$T(n) = a\,T(n/b) + f(n), \qquad T(1) = \Theta(1), \tag{3.1}$$

with $a \geq 1$, $b > 1$ constants and f asymptotically positive. We assume n is an exact power of b; floors and ceilings change bounds by at most a constant factor and are handled by standard domain-transformation arguments.

Running examples. MergeSort: $T(n) = 2T(n/2) + \Theta(n)$. Karatsuba integer multiplication (Section 3.4.1): $T(n) = 3T(n/2) + \Theta(n)$, where n counts digits or bits. Binary search: $T(n) = T(n/2) + \Theta(1)$. Strassen matrix multiplication: $T(n) = 7T(n/2) + \Theta(n^2)$.

Our goal is a Θ-bound for $T(n)$. We present methods in order of increasing automation—and decreasing generality.

Remark 3.1 (Floors, ceilings, and base cases). Replacing $T(n/b)$ by $T(\lfloor n/b \rfloor)$ or $T(\lceil n/b \rceil)$ does not change the asymptotic answer for the recurrences in this chapter: one shows $T'(n) \leq T(n) \leq T''(n)$ for suitable exact-power variants T', T'' that differ by at most a constant factor. Base cases $T(1) = \Theta(1)$ vs. $T(n) = O(1)$ for $n \leq n_0$ are similarly interchangeable up to constants; only the inductive step matters for asymptotics.

3.2 The Substitution Method

Guess a bound, then verify by induction. The method is fully general but requires insight to produce the right guess; recursion trees (next section) are the usual source of guesses.

Example 3.1 (MergeSort upper bound)**.** Claim $T(n) = 2T(n/2) + n$ with $T(1) = 1$ satisfies $T(n) = O(n \log n)$. Guess $T(n) \leq cn \log n$ for $c > 0$ and $n \geq 2$. Assume it holds for $n/2$:

$$T(n) \leq 2c\frac{n}{2}\log\frac{n}{2} + n = cn\log n - cn + n \leq cn\log n$$

provided $c \geq 1$. Choosing c large enough to handle base cases completes the induction. A symmetric argument gives $T(n) \geq c'n \log n$, so $T(n) = \Theta(n \log n)$.

Example 3.2 (Strengthening the hypothesis)**.** To prove $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$ is $O(n)$, the naive guess $T(n) \le cn$ fails because $T(n) \le c\lfloor n/2 \rfloor + c\lceil n/2 \rceil + 1 = cn + 1$. Strengthen to $T(n) \le cn - b$ for constants $c, b > 0$:

$$T(n) \le c\lfloor n/2 \rfloor - b + c\lceil n/2 \rceil - b + 1 = cn - 2b + 1 \le cn - b$$

once $b \ge 1$. Subtracting a low-order term to absorb the additive constant is a standard trick.

When to use it: substitution is the only method that proves matching upper and lower bounds with full rigour, including floors. *Pitfalls:* the guess must be asymptotically tight; inductive algebra with floors needs careful bounding ($n/2 - 1 < \lfloor n/2 \rfloor \le n/2$).

Change of variables. Some recurrences do not fit (3.1) directly. For $T(n) = 2T(\sqrt{n}) + \log n$, set $m = \log n$ and $S(m) = T(2^m)$ to obtain $S(m) = 2S(m/2) + m$, i.e. MergeSort in m, so $S(m) = \Theta(m \log m)$ and $T(n) = \Theta(\log n \cdot \log\log n)$. The same idea handles $T(n) = T(n/2) + \Theta(\log n)$-type recurrences after a substitution $n = 2^k$.

3.3 The Recursion-Tree Method

Expand the recurrence into a tree, sum level by level, then verify the resulting guess by substitution. The tree is a discovery tool; the substitution step remains the proof.

For (3.1), level 0 has cost $f(n)$, level 1 has a nodes each costing $f(n/b)$, and level i has a^i nodes each costing $f(n/b^i)$. If the tree has depth $L = \log_b n$,

$$T(n) = \sum_{i=0}^{L-1} a^i f(n/b^i) + \Theta(n^{\log_b a}), \tag{3.2}$$

where the last term is the total leaf cost ($a^L = n^{\log_b a}$ leaves each costing $\Theta(1)$). Bounding this sum is the entire problem. Figure 3.1 illustrates the expansion.

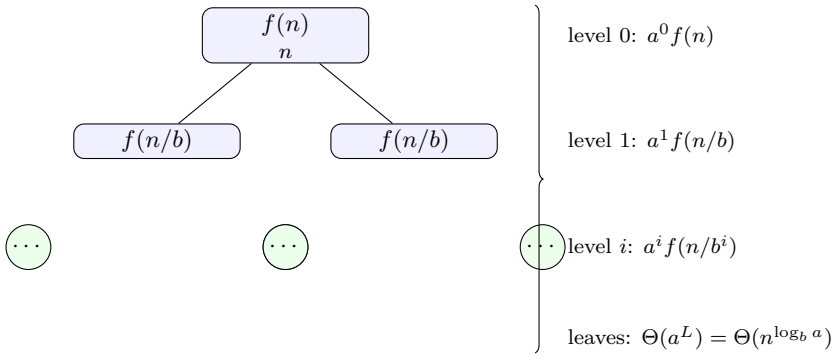


Figure 3.1: Recursion tree for $T(n) = aT(n/b) + f(n)$. Total cost is the sum over levels plus leaves; $L = \log_b n$.

Example 3.3 (MergeSort via tree)**.** $T(n) = 2T(n/2) + cn$. Level i has 2^i nodes of cost $cn/2^i = cn$, so each level costs cn. With $\log_2 n$ levels, $T(n) = cn \log_2 n + \Theta(n) = \Theta(n \log n)$.

Example 3.4 (Karatsuba preview)**.** $T(n) = 3T(n/2) + cn$. Level i costs $3^i \cdot c(n/2^i) = cn(3/2)^i$—a geometric series with ratio $3/2 > 1$, dominated by its last term:

$$\sum_{i=0}^{L-1} cn(3/2)^i = cn\frac{(3/2)^L - 1}{3/2 - 1} = \Theta(n \cdot n^{\log_2 3 - 1}) = \Theta(n^{\log_2 3}).$$

The Master theorem automates this calculation.

Remark 3.2. Trees also handle recurrences outside the Master form, e.g. $T(n) = T(n/3) + T(2n/3) + \Theta(n)$, where the longest root-to-leaf path has length $\log_{3/2} n$ and a careful level-by-level bound still yields $\Theta(n \log n)$—a case the Akra–Bazzi method settles directly. In general, bound the sum (3.2) by comparing it to a geometric series: if $a^i f(n/b^i)$ grows (resp. shrinks) by a constant factor each level, the sum is dominated by the last (resp. first) term.

Recurrence with non-constant work. Consider $T(n) = 2T(n/2) + n \log n$. Level i has 2^i nodes each of cost $(n/2^i)\log(n/2^i)$, so level cost is $n(\log n - i)$. Summing gives $n \sum_{i=0}^{L-1} (\log n - i) = \Theta(n \log^2 n)$, which matches Master Case 2 with $k = 1$.

3.4 The Master Theorem

The Master theorem classifies (3.1) when f is polynomially comparable to $n^{\log_b a}$, the leaf-driven rate. It replaces tree summation with a case check.

Theorem 3.1 (Master theorem)**.** Let $a \geq 1$, $b > 1$, f asymptotically positive, and $T(n) = aT(n/b) + f(n)$. Let $c_\star = \log_b a$.

Case 1: If $f(n) = O(n^{c_\star - \varepsilon})$ for some $\varepsilon > 0$, then $T(n) = \Theta(n^{c_\star})$.

Case 2: If $f(n) = \Theta(n^{c_\star} \log^k n)$ for some $k \geq 0$, then $T(n) = \Theta(n^{c_\star} \log^{k+1} n)$. In particular $f(n) = \Theta(n^{c_\star})$ gives $\Theta(n^{c_\star} \log n)$.

Case 3: If $f(n) = \Omega(n^{c_\star + \varepsilon})$ for some $\varepsilon > 0$ and the *regularity* condition $af(n/b) \leq cf(n)$ holds for some $c < 1$ and large n, then $T(n) = \Theta(f(n))$.

Figure 3.2 gives the geometric intuition: the three cases correspond to whether per-level costs $a^i f(n/b^i)$ in (3.2) shrink, stay constant, or grow geometrically as i increases.

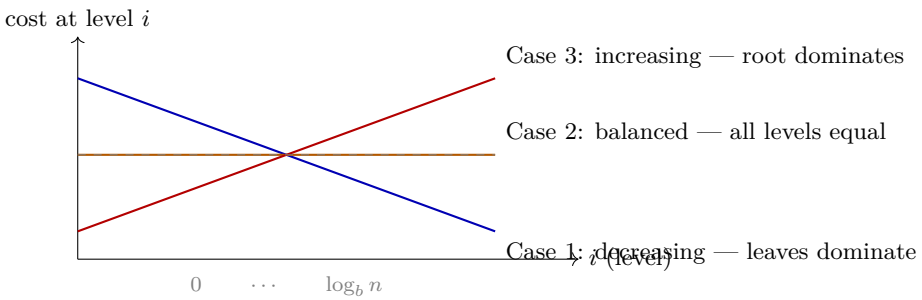


Figure 3.2: Master theorem geometry: per-level cost $a^i f(n/b^i)$ across tree levels. The dominant contribution determines the case.

Proof sketch. Write $g(n) = f(n)/n^{c_\star}$ so that $a^i f(n/b^i) = n^{c_\star} g(n/b^i)$. Then (3.2) becomes $T(n) = n^{c_\star} \sum_{i=0}^{L-1} g(n/b^i) + \Theta(n^{c_\star})$. In Case 1, $g(n) = O(n^{-\varepsilon})$ and the sum is a convergent geometric series bounded by a constant, so $T(n) = \Theta(n^{c_\star})$. In Case 2, $g(n) = \Theta(\log^k n)$ and $g(n/b^i) = \Theta((\log n - i \log b)^k)$, whose sum over $L = \Theta(\log n)$ terms contributes $\Theta(\log^{k+1} n)$. In Case 3, regularity gives $a^i f(n/b^i) \leq c^i f(n)$, so level costs decay geometrically toward the leaves and $\sum a^i f(n/b^i) = \Theta(f(n))$. Floors and ceilings add only lower-order terms.

Why the regularity condition is needed. Without it Case 3 is false: there exist (contrived) f with $f(n) = \Omega(n^{c_\star + \varepsilon})$ but $T(n) \neq \Theta(f(n))$ because f dips sharply at infinitely many n/b^i, breaking the geometric decay. For polynomials and most natural functions the condition is automatic—check $af(n/b)/f(n) = a/b^k < 1$ when $f(n) = n^k$ with $k > c_\star$.

Regularity explained. The condition $af(n/b) \le cf(n)$ for $c < 1$ says work shrinks by a constant factor each level going up the tree, guaranteeing root domination. It holds for all polynomials $f(n) = n^k$ with $k > c_\star$ and for $f(n) = n^{c_\star+\varepsilon}/\log n$, but can fail for contrived oscillating f such as $f(n) = n^{c_\star}(2 + \sin n)$, where no single $c < 1$ works for all large n.

Gaps. The theorem does *not* cover every f. The polynomial gap $\varepsilon > 0$ is essential: $f(n) = n^{c_\star} \log\log n$ and $f(n) = n^{c_\star}/\log n$ fall strictly between Cases 1 and 2—no ε separates them from $n^{c_\star}$. Likewise $f(n) = n^{c_\star} \sin n$ violates regularity. These require the tree method or Akra–Bazzi. The extended Case 2 above ($\log^k n$ factor) is the most common gap students encounter; it is included in Theorem 3.1 but not in all textbook statements.

How to apply it in practice. (1) Compute $c_\star = \log_b a$. (2) Compare $f(n)$ to $n^{c_\star}$: is it polynomially smaller, Theta-equivalent (up to $\log^k n$), or polynomially larger with regularity? (3) If none holds, the Master theorem is silent—use a tree or Akra–Bazzi. Common mistake: claiming Case 3 without checking regularity, or ignoring the ε requirement and misclassifying $n^{c_\star} \log n$ as Case 1 or 3.

3.4.1 Worked Applications

MergeSort. $a = 2$, $b = 2$, $c_\star = 1$, $f(n) = \Theta(n) = \Theta(n^{c_\star})$: Case 2 gives $T(n) = \Theta(n \log n)$.

Karatsuba. Write n-digit numbers as $X = a \cdot 10^m + b$, $Y = c \cdot 10^m + d$ with $m = n/2$. Naively XY needs four half-products; Karatsuba uses ac, bd, and $(a - b)(c - d)$ to recover $ad + bc$ with three products: $T(n) = 3T(n/2) + \Theta(n)$. Here $c_\star = \log_2 3 \approx 1.585$ and $f(n) = O(n^{c_\star - 0.58})$: Case 1 gives $T(n) = \Theta(n^{\log_2 3})$, strictly beating the naive $\Theta(n^2)$.

Strassen. $T(n) = 7T(n/2) + \Theta(n^2)$: $c_\star = \log_2 7 \approx 2.807 > 2$, Case 1 gives $\Theta(n^{\log_2 7})$.

Binary search. $T(n) = T(n/2) + \Theta(1)$: $c_\star = 0$, $f(n) = \Theta(1) = \Theta(n^{c_\star})$, Case 2 gives $\Theta(\log n)$.

Quick check. $T(n) = 9T(n/3) + \Theta(n^2)$: $c_\star = 2$, $f(n) = \Theta(n^{c_\star})$ so Case 2 gives $\Theta(n^2 \log n)$. Changing to $f(n) = n^3$ switches to Case 3 ($\Theta(n^3)$), while $f(n) = n$ switches to Case 1 ($\Theta(n^2)$). The exponent of f relative to $c_\star$ is decisive.

Relation of $f(n)$ to $n^{c_\star}$	Case	$T(n)$	Intuition
$f(n) = O(n^{c_\star - \varepsilon})$	1	$\Theta(n^{c_\star})$	Leaves dominate
$f(n) = \Theta(n^{c_\star} \log^k n)$	2	$\Theta(n^{c_\star} \log^{k+1} n)$	Balanced levels
$f(n) = \Omega(n^{c_\star + \varepsilon})$, regular	3	$\Theta(f(n))$	Root dominates
Gaps: $n^{c_\star}/\log n$, $n^{c_\star} \log\log n$	—	—	Use tree / Akra–Bazzi

Table 3.1: Master theorem at a glance ($c_\star = \log_b a$). The table is a memory aid, not a replacement for checking ε and regularity.

3.5 Beyond Equal Subproblems: Akra–Bazzi

When subproblem sizes differ or f is not polynomially separated, the Master theorem does not apply. The Akra–Bazzi theorem handles

$$T(n) = \sum_{i=1}^{k} a_i T(b_i n + h_i(n)) + f(n), \quad 0 < b_i < 1,\ a_i > 0,$$

with perturbations $h_i(n) = O(n/\log^2 n)$. Let p be the unique solution of $\sum_{i=1}^{k} a_i b_i^p = 1$. Then

$$T(n) = \Theta\Big(n^p\Big(1 + \int_1^n \frac{f(u)}{u^{p+1}}\,du\Big)\Big).$$

For the Master form $aT(n/b) + f(n)$ this recovers all three cases. For $T(n) = T(n/3) + T(2n/3) + \Theta(n)$ we get $p = 1$ and $\int_1^n u^{-1}du = \log n$, so $T(n) = \Theta(n \log n)$. Another check: $T(n) = 2T(n/4) + \Theta(\sqrt{n})$ has $2(1/4)^p = 1 \Rightarrow p = 1/2$, and $\int_1^n u^{1/2}/u^{3/2}du = \log n$, so $T(n) = \Theta(\sqrt{n}\log n)$, matching Master Case 2. We state the result without proof; the integral replaces the polynomial-gap check.

Remark 3.3 (Using Akra–Bazzi in SC2001). In this course you may quote Akra–Bazzi (and the Master theorem) without reproving it, but you must verify its hypotheses: $a_i > 0$, $0 < b_i < 1$, the perturbation $h_i(n) = O(n/\log^2 n)$, and f polynomially bounded. For $T(n) = T(n/2) + T(n/4) + \Theta(n)$ the equation $(1/2)^p + (1/4)^p = 1$ gives $p \approx 0.694$; the integral is $\Theta(n)$-dominated, yielding $T(n) = \Theta(n)$. This refines the tree estimate and illustrates why the Master theorem alone is insufficient when subproblem sizes differ.

3.6 Chapter Notes and Exercises

Substitution builds proof discipline; trees build intuition; the Master theorem automates the common case. Table 3.1 summarises the cases. When subproblems are unequal, floors are messy, or f oscillates, fall back to trees or Akra–Bazzi. A useful habit: conjecture with a tree, confirm the dominant case with the Master theorem when it applies, and verify the final bound by substitution. Exercises below test each method and the gaps between them.

Exercise 3.1 (Substitution practice)**.** Solve $T(n) = 4T(n/2) + n$ with $T(1) = 1$. Guess $T(n) = O(n^2)$ and prove it by substitution. Then prove a matching $\Omega(n^2)$ bound to conclude $T(n) = \Theta(n^2)$. Which Master case does this correspond to?

Exercise 3.2 (Recursion tree)**.** Use a recursion tree to conjecture tight bounds for (a) $T(n) = 2T(n/2) + n^2$ and (b) $T(n) = T(n/2) + T(n/4) + \Theta(n)$. Verify one of them by substitution. For which, if any, does the Master theorem apply directly?

Exercise 3.3 (Master theorem gaps)**.** For each recurrence state which Master case applies (or why none does) and give the resulting bound when applicable: (a) $T(n) = 2T(n/2) + n\log n$, (b) $T(n) = 2T(n/2) + n/\log n$, (c) $T(n) = 4T(n/2) + n^2\log n$, (d) $T(n) = 2T(n/2) + \Theta(n^{1.5})$.

Exercise 3.4 (Karatsuba vs. naive)**.** Karatsuba achieves $T_K(n) = 3T_K(n/2) + \Theta(n)$ while naive divide-and-conquer needs $T_N(n) = 4T_N(n/2) + \Theta(n)$. Compute both bounds and find the smallest n (as a power of 2) where $n^{\log_2 3} < n^2/10$ under leading constants 1, to quantify the asymptotic crossover.

Chapter 4

Divide and Conquer

> *"Divide et impera."* — The oldest strategy in warfare is also one of the most powerful in algorithm design: break a hard problem into smaller copies of itself, solve them recursively, and combine.

Prerequisites: Recurrences and asymptotics (Ch. 2), induction and correctness (Ch. 1), basic probability (linearity of expectation, indicator variables).

Learning Outcomes

After this chapter you will be able to:

(L1) state the D&C paradigm and write recurrences for D&C algorithms;

(L2) prove correctness and $\Theta(n \log n)$ complexity of MergeSort;

(L3) analyse deterministic QuickSort's worst case and randomised QuickSort's expected $O(n \log n)$ via indicator variables;

(L4) prove the strip sparsity lemma for the closest-pair problem;

(L5) derive Strassen's 7-multiplication recurrence and its $O(n^{\log_2 7})$ bound.

4.1 The Divide-and-Conquer Paradigm

A D&C algorithm for input size n does three steps:

1. **Divide** the instance into $a \geq 1$ subinstances each of size $\lceil n/b \rceil$ or $\lfloor n/b \rfloor$ ($b > 1$).
2. **Conquer** by solving subinstances recursively (base case $n \leq n_0$ solved directly).
3. **Combine** sub-solutions into a solution for the original instance at cost $f(n)$.

Hence

$$T(n) = a\,T(n/b) + f(n), \qquad T(1) = \Theta(1). \tag{4.1}$$

The Master Theorem (Ch. 2) classifies (4.1): if $f(n) = O(n^{\log_b a - \epsilon})$ then $T(n) = \Theta(n^{\log_b a})$; if $f(n) = \Theta(n^{\log_b a})$ then $T(n) = \Theta(n^{\log_b a} \log n)$; if $f(n) = \Omega(n^{\log_b a + \epsilon})$ and regular then $T(n) = \Theta(f(n))$.

D&C succeeds when the combine step is cheap relative to the progress made by shrinking subproblems. The next sections are canonical examples.

4.2 MergeSort

`MergeSort`$(A[p..r])$: if $p < r$, let $q = \lfloor (p+r)/2 \rfloor$, recursively sort $A[p..q]$ and $A[q+1..r]$, then `Merge` the two sorted halves in $\Theta(n)$ time using an auxiliary array.

Theorem 4.1 (Correctness of MergeSort)**.** `MergeSort` returns a permutation of the input sorted in nondecreasing order.

Proof. Induction on $n = r - p + 1$. Base $n = 1$ trivially sorted. Assume holds for all sizes $< n$. Both halves are sorted by the induction hypothesis. `Merge` repeatedly takes the smaller leading element of the two sorted halves, so the output is sorted and contains exactly the elements of $A[p..r]$. By induction the claim holds for n. □

`Merge` scans each element once, so $f(n) = \Theta(n)$. Thus

$$T(n) = 2T(n/2) + \Theta(n), \quad T(1) = \Theta(1).$$

Here $a = 2$, $b = 2$, $\log_b a = 1$, $f(n) = \Theta(n) = \Theta(n^{\log_b a})$, so Master case 2 gives

$$T(n) = \Theta(n \log n).$$

This is asymptotically optimal for comparison sorting (lower bound in Ch. 8). Unlike QuickSort, MergeSort guarantees $\Theta(n \log n)$ worst-case but needs $\Theta(n)$ extra space.

4.3 QuickSort

`Partition`(A, p, r) picks pivot $x = A[r]$, maintains index i with $A[p..i] \le x$ and $A[i+1..j-1] > x$, scanning j from p to $r-1$ and swapping when $A[j] \le x$. Finally swap $A[i+1]$ with $A[r]$; return $q = i + 1$. Cost is $\Theta(n)$.

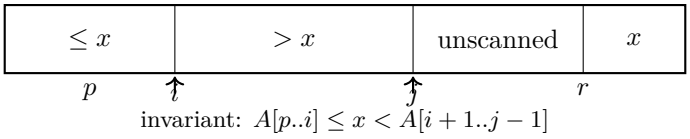


Figure 4.1: Lomuto partition invariant. Elements left of i are $\le$ pivot; between $i+1$ and $j-1$ are $>$ pivot.

`QuickSort`(A, p, r): if $p < r$, $q \leftarrow$`Partition`(A, p, r), recurse on $A[p..q-1]$ and $A[q+1..r]$.

Deterministic worst case. If the pivot is always the smallest or largest element (e.g., sorted input with $A[r]$ as pivot), partitions are 0 and $n-1$:

$$T(n) = T(n-1) + \Theta(n) = \Theta\left(\sum_{k=1}^{n} k\right) = \Theta(n^2).$$

Balanced partitions ($n/4$–$3n/4$) still give $T(n) \le 2T(3n/4) + \Theta(n) = \Theta(n \log n)$, but the adversary can force $\Theta(n^2)$.

Randomised QuickSort. Choose pivot uniformly at random from $A[p..r]$ (or randomly permute first). Let $z_1 < \cdots < z_n$ be sorted values and $X_{ij} = 1$ if z_i and z_j are compared (i.e., one is chosen as pivot while both lie in the same recursive call), else 0. Total comparisons $X = \sum_{i<j} X_{ij}$.

For $i < j$, z_i and z_j are compared iff among $Z_{ij} = \{z_i, \ldots, z_j\}$ the first element chosen as pivot is z_i or z_j. All $|Z_{ij}| = j - i + 1$ elements are equally likely to be chosen first, so

$$\Pr[X_{ij} = 1] = \frac{2}{j - i + 1}, \qquad \mathbb{E}[X_{ij}] = \frac{2}{j - i + 1}.$$

By linearity of expectation,

$$\mathbb{E}[X] = \sum_{i=1}^{n} \sum_{j=i+1}^{n} \frac{2}{j - i + 1} = \sum_{i=1}^{n} \sum_{k=1}^{n-i} \frac{2}{k + 1} < 2 \sum_{i=1}^{n} H_n = O(n \log n),$$

more precisely $\mathbb{E}[X] = 2(n + 1)H_n - 4n \sim 2n \ln n$. Each comparison is $O(1)$ work, so

$$\mathbb{E}[T(n)] = O(n \log n).$$

Randomisation thus removes the adversary's power: *every* input has expected $O(n \log n)$ time.

4.4 Closest Pair of Points

Given $P = \{p_1, \ldots, p_n\} \subset \mathbb{R}^2$, find $\min_{i \neq j} \|p_i - p_j\|_2$. Brute force is $\Theta(n^2)$. The D&C algorithm sorts by x, splits by median $x = m$ into P_L, P_R, recursively finds δ_L, δ_R, sets $\delta = \min(\delta_L, \delta_R)$, then checks pairs straddling the dividing line within the vertical strip $|x - m| < \delta$.

Naively the strip could contain $\Theta(n)$ points. The key is sparsity:

Lemma 4.1 (Strip sparsity)**.** Let points in the strip be sorted by y. Any $\delta \times 2\delta$ rectangle in the strip contains at most 6 points of P (at most 4 with a tighter packing argument gives 7 total neighbours to check).

Proof. Partition the rectangle into 6 (or 8) squares of side $\delta/2$. The diagonal is $\delta/\sqrt{2} < \delta$. If two points lay in the same square their distance would be $< \delta$, contradicting that inter-point distances within P_L and P_R are $\geq \delta$. So at most one point per square. □

Consequently, for each point in the strip sorted by y, only the next 7 points need be checked. The strip scan is $O(n)$ after $O(n \log n)$ presorting (or $O(n)$ with presorted arrays passed down). Recurrence:

$$T(n) = 2T(n/2) + O(n) \implies T(n) = O(n \log n).$$

This beats $\Theta(n^2)$ and is optimal in the algebraic decision-tree model. In practice the y-sorted order is maintained by presorting once ($O(n \log n)$) and passing the y-order through recursion, so no per-level sort is needed.

Remark 4.1. The closest-pair recurrence is identical in form to MergeSort's; the difficulty is entirely in the combine step. The sparsity lemma is what keeps $f(n) = O(n)$.

4.5 Strassen's Matrix Multiplication

Naive $n \times n$ multiplication costs $\Theta(n^3)$. Partition

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \; B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \; C = AB = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix},$$

each block $n/2 \times n/2$. Standard block multiplication needs 8 half-size products. Strassen (1969) uses 7:

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22}) \qquad M_5 = (A_{11} + A_{12})B_{22}$$
$$M_2 = (A_{21} + A_{22})B_{11} \qquad M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$
$$M_3 = A_{11}(B_{12} - B_{22}) \qquad M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$
$$M_4 = A_{22}(B_{21} - B_{11})$$

Then $C_{11} = M_1 + M_4 - M_5 + M_7$, $C_{12} = M_3 + M_5$, $C_{21} = M_2 + M_4$, $C_{22} = M_1 - M_2 + M_3 + M_6$. Each M_k is one $n/2 \times n/2$ product; the 18 additions cost $\Theta(n^2)$.

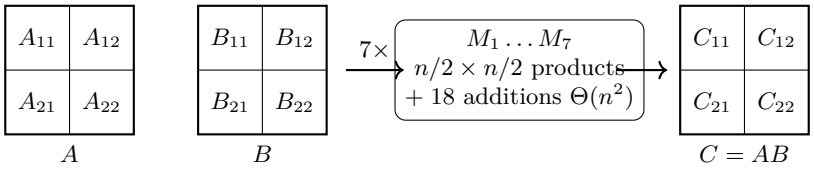


Figure 4.2: Strassen: one $n \times n$ product reduced to seven $n/2 \times n/2$ products plus $O(n^2)$ additions.

Recurrence:

$$T(n) = 7\,T(n/2) + \Theta(n^2), \quad T(1) = \Theta(1).$$

Here $\log_b a = \log_2 7 \approx 2.807$ and $f(n) = O(n^2) = O(n^{\log_2 7 - \epsilon})$, so Master case 1 gives

$$T(n) = \Theta(n^{\log_2 7}) = O(n^{2.81}),$$

asymptotically faster than $\Theta(n^3)$. Applied recursively (padding to power of two if needed), it is practical for large n despite larger constants.

4.6 Exercises

4.1 MergeSort invariants. Write the loop invariant for `Merge`(L, R) that merges two sorted arrays of total length n. Prove initialization, maintenance, and termination, and show the number of comparisons is at most $n - 1$.

4.2 QuickSort indicators. Let X_{ij} be as in §4.3. (a) Justify $\Pr[X_{ij} = 1] = 2/(j - i + 1)$ by symmetry among Z_{ij}. (b) Compute $\mathbb{E}[X]$ exactly as $2(n + 1)H_n - 4n$ and deduce $\mathbb{E}[X] \le 2n \ln n + O(n)$. What happens if pivots are chosen deterministically as $A[r]$ on a random permutation input?

4.3 Closest-pair tightness. (a) Prove the strip lemma with 8 squares of side $\delta/2$ implies at most 7 successors need be checked. (b) Give a configuration of 8 points achieving 6 points in a $\delta \times 2\delta$ rectangle with all pairwise distances $\ge \delta$, showing the constant 7 cannot be reduced to 3 without a finer argument.

4.4 Strassen in practice. (a) Verify algebraically that the seven M_k yield the correct C_{ij}. (b) Show that for $n = 2^k$ the exact operation count satisfies $T(n) = 7^k + 6 \cdot \sum_{i=0}^{k-1} 7^i 4^{k-1-i}$ and confirm $T(n) = \Theta(n^{\log_2 7})$. At what n_0 would you switch to naive multiplication if Strassen's constant is c_s vs. c_0?

Chapter 5

Greedy Algorithms

> *"Make the best local choice and never look back."* — The greedy motto. Sometimes it yields optimal solutions; often it does not. The craft lies in knowing which and proving it.

Prerequisites: Asymptotic notation (Ch. 2), exchange arguments and induction (Ch. 1), priority queues and Union-Find (Ch. 3), basic graph terminology (Ch. 6 preview).

Learning Outcomes

After this chapter you will be able to:

(L1) describe the greedy paradigm and state the greedy-choice and optimal-substructure conditions;

(L2) apply the exchange-argument template to prove correctness of a greedy algorithm;

(L3) prove optimality of earliest-finish-time interval scheduling;

(L4) construct Huffman codes and prove their optimality;

(L5) state and apply the cut property to justify Kruskal's and Prim's MST algorithms;

(L6) exhibit inputs where the greedy heuristic fails (coin change).

5.1 The Greedy Paradigm and the Exchange Argument

A *greedy* algorithm builds a solution incrementally, choosing at each step the locally best option according to some criterion, never reconsidering earlier choices.

Definition 5.1 (Greedy choice & optimal substructure)**.** An optimisation problem has the *greedy-choice property* if a globally optimal solution can be assembled by making a locally optimal first choice and then solving the remaining subproblem optimally. It has *optimal substructure* if an optimal solution contains optimal solutions to subproblems.

Greedy is not universally correct; it requires both properties. Proofs of correctness almost always use an *exchange argument*:

> **Exchange template.** Let G be the greedy solution and O an optimal solution that agrees with G for as long as possible (first k choices). Show the $(k{+}1)$st greedy choice can be exchanged into O without worsening its value, obtaining O' that agrees with G one step further and remains optimal. By induction G is optimal.

Two recurring lemmas support the exchange: (i) *greedy stays ahead* (after k steps the greedy partial solution is at least as good as any competitor's), and (ii) *cut/exchange* (swapping in the greedy element does not hurt feasibility or cost).

5.2 Interval Scheduling: Earliest Finish Time

Given n intervals $[s_i, f_i)$ (half-open), find a maximum-cardinality subset of pairwise compatible (non-overlapping) intervals.

```
EarliestFinish(intervals):
  sort by finish time f_1 <= f_2 <= ... <= f_n
  S = empty; last = -inf
  for i = 1..n:
    if s_i >= last:  S.add(i); last = f_i
  return S
```

Theorem 5.1 (Optimality of earliest finish)**.** EarliestFinish returns a maximum-size compatible set.

Exchange argument. Let $G = \langle g_1, \ldots, g_k \rangle$ be the greedy set in order of finish time and $O = \langle o_1, \ldots, o_m \rangle$ an optimal set ordered the same way, chosen to maximise the longest common prefix with G. Let r be the first index where they differ (or $r = k + 1$ if G is a prefix of O). If $r > k$ then $|G| \le |O|$ and, since G is maximal, $|G| = |O|$. Otherwise $f_{g_r} \le f_{o_r}$ (greedy picks the earliest-finishing compatible interval) and $s_{o_{r+1}} \ge f_{o_r} \ge f_{g_r}$, so $O' = (O \setminus \{o_r\}) \cup \{g_r\}$ is feasible and $|O'| = |O|$. O' is optimal and agrees with G one step further, contradicting maximality of r. Hence no such r exists and $|G| = |O|$. □

The argument shows *greedy stays ahead*: $f_{g_i} \le f_{o_i}$ for all $i \le \min(k, m)$. This invariant is the core of many greedy proofs—compare the kth greedy choice directly to the kth choice of any competitor.

Example 5.1 (Small instance)**.** Intervals $[1, 4), [3, 5), [0, 6), [5, 7), [8, 9)$ sorted by finish time are $[1, 4), [3, 5), [0, 6), [5, 7), [8, 9)$. Greedy picks $[1, 4), [5, 7), [8, 9)$ (size 3), which is optimal. Replacing $[1, 4)$ by $[0, 6)$ cannot improve the count.

Running time is $O(n \log n)$ for sorting plus $O(n)$ scanning. Variants (earliest start time, shortest interval, fewest conflicts) all fail—counterexamples are left as exercises.

5.3 Huffman Coding

Given alphabet C with frequencies $freq(c) > 0$, find a binary *prefix code* (no codeword is a prefix of another) minimising weighted path length $B(T) = \sum_c freq(c) \cdot d_T(c)$, where $d_T(c)$ is depth in the coding tree T.

Huffman's greedy rule: repeatedly merge the two lowest-frequency nodes.

```
Huffman(C, freq):
  Q = min-heap of leaves by freq
  for i = 1 to |C|-1:
    x = Q.extractMin(); y = Q.extractMin()
    z = internal node with children x,y, freq(z)=freq(x)+freq(y)
    Q.insert(z)
  return Q.extractMin()  // root
```

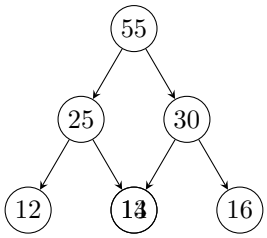


Figure 5.1: Huffman tree sketch: internal labels are summed frequencies; leaves carry character frequencies (here $12, 13, 14, 16$). The two smallest $(12, 13)$ are siblings at maximum depth—the greedy choice.

Lemma 5.1 (Sibling property)**.** There exists an optimal prefix tree in which the two least-frequent characters are siblings at maximum depth.

Exchange. Take an optimal tree T and let x, y be the two least-frequent characters. Let a, b be siblings at maximum depth in T. If $\{x, y\} = \{a, b\}$ we are done. Otherwise swap x with a (and y with b if needed). Since $freq(x) \leq freq(a)$ and $d(x) \leq d(a)$ cannot both be reversed, each swap does not increase $B(T)$: $\Delta = (freq(a) - freq(x))(d(a) - d(x)) \geq 0$ cost is not raised. The resulting tree is optimal with x, y as deepest siblings. □

Theorem 5.2 (Huffman optimality)**.** Huffman produces an optimal prefix code.

Induction + exchange. Induction on $|C| = n$. Base $n = 2$ trivial. For $n > 1$, let x, y be the two smallest frequencies merged into z with $freq(z) = freq(x) + freq(y)$. Let T' be the optimal tree for $C' = (C \setminus \{x, y\}) \cup \{z\}$ given by induction (Huffman on C'). Expand z into x, y to get T; then $B(T) = B(T') + freq(x) + freq(y)$. If some tree T^* for C had $B(T^*) < B(T)$, transform T^* by the Lemma so x, y are siblings, contract them to z to get T_0^* for C' with $B(T_0^*) = B(T^*) - freq(x) - freq(y) < B(T')$, contradicting optimality of T'. □

Complexity is $O(n \log n)$ with a heap. Decoding is unambiguous precisely because the code is prefix-free—the tree path is followed until a leaf is reached. Optimal merging and the structure of prefix codes both hinge on the same exchange idea.

Remark 5.1. Kraft's inequality $\sum_c 2^{-d_T(c)} \leq 1$ characterises feasible prefix-code lengths. Huffman finds lengths satisfying it with minimum $\sum freq(c) d_T(c)$, i.e. minimum redundancy above entropy $H = -\sum p_c \log p_c$.

5.4 Minimum Spanning Trees: The Cut Property

Let $G = (V, E)$ be connected, undirected, with distinct edge weights $w(e)$ (ties broken arbitrarily). A *spanning tree* minimises total weight among all spanning trees if it is an MST.

Definition 5.2 (Cut property)**.** For any cut $(S, V \setminus S)$, the minimum-weight edge crossing the cut belongs to every MST.

Exchange. Let $e = (u, v)$ be the lightest edge crossing $(S, V \setminus S)$ and suppose some MST T omits e. Adding e to T creates a unique cycle C, which must cross the cut in another edge $f \neq e$. Since $w(e) < w(f)$, $T' = (T \setminus \{f\}) \cup \{e\}$ is a spanning tree lighter than T—contradiction. Hence $e \in T$. □

Both classic algorithms are greedy instantiations of this property.

```
Kruskal(V,E,w):
  sort E by w ascending; make Union-Find on V
  T = empty
  for e=(u,v) in sorted E:
    if Find(u) != Find(v): T.add(e); Union(u,v)
  return T
```

Each edge added by Kruskal is the lightest crossing the cut $(C, V \setminus C)$ where C is its component before the union.

```
Prim(V,E,w, r):
  S={r}; T=empty; PQ = edges leaving S keyed by w
  while S != V:
    e=(u,v)=PQ.extractMin() with u in S, v not in S
    T.add(e); S.add(v); insert edges from v to V\S into PQ
  return T
```

Each edge added by Prim is exactly the minimum edge of cut $(S, V \setminus S)$.

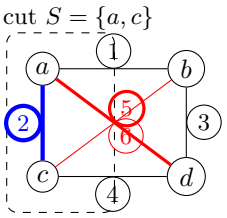


Figure 5.2: Cut $(S, V \setminus S)$ with $S = \{a, c\}$. The lightest crossing edge (weight 1, a–b) must be in every MST; any MST containing a heavier crossing edge can be improved by exchange.

Correctness of both follows immediately from the cut property: every edge they add is forced into every MST, so the final tree is contained in all MSTs and hence is one itself. Kruskal runs in $O(|E| \log |E|)$ (sorting dominates); Prim with a binary heap is $O(|E| \log |V|)$, $O(|E| + |V| \log |V|)$ with a Fibonacci heap. The cycle property (heaviest edge on any cycle is never in any MST) is dual and also proved by exchange.

5.5 When Greedy Fails: Coin Change

Given denominations $D = \{d_1 < \cdots < d_k\}$ and amount A, minimise number of coins. The greedy rule—repeatedly take the largest $d_i \le$ remaining amount—is optimal for canonical systems (e.g. US coins $\{1, 5, 10, 25\}$) but not in general.

Example 5.2 (Counterexample)**.** $D = \{1, 3, 4\}$, $A = 6$. Greedy picks $4 + 1 + 1$ (3 coins) while $3 + 3$ uses 2 coins. For $D = \{1, 10, 25\}$, $A = 30$: greedy gives $25 + 1 + 1 + 1 + 1 + 1$ (6 coins) vs. $10 + 10 + 10$ (3 coins).

The DP table for $D = \{1, 3, 4\}$, $A = 6$ illustrates the fix:

x	0	1	2	3	4	5	6
$dp[x]$	0	1	2	1	1	2	2
choice	–	1	1	3	4	1+4	3+3

Greedy is myopic: taking 4 at $x = 6$ forces two 1s, while dp considers all predecessors $x - d_i$.

More generally, no fixed greedy order works for all D. The problem is solved optimally by dynamic programming in $O(kA)$:

$$dp[0] = 0, \quad dp[x] = \min_{d_i \le x}\{1 + dp[x - d_i]\}.$$

The failure illustrates the missing greedy-choice property: a locally optimal coin can block a globally better combination.

5.6 Exercises

E1. **Interval variants that fail.** Give counterexamples showing that (a) earliest start time, (b) shortest interval first, and (c) fewest conflicts first are *not* optimal for interval scheduling.

E2. **Huffman trace.** For frequencies $\{5, 9, 12, 13, 16, 45\}$, simulate Huffman step-by-step, draw the final tree, compute $B(T)$, and verify the average code length against the entropy lower bound.

E3. **Coin-change classification.** Prove that greedy is optimal for $D = \{1, 5, 10, 25\}$ (US coins without the half-dollar). Then find the smallest A where greedy fails for $D = \{1, 3, 4\}$ and prove dp above is correct.

E4. **Cut/cycle duality.** Prove the *cycle property*: if e is the maximum-weight edge on some cycle, no MST contains e. Deduce that the MST is unique when all edge weights are distinct.

E5. **Scheduling with deadlines.** Each job has duration t_i and deadline d_i. The greedy order by earliest deadline minimises maximum lateness $L_{\max} = \max_i(f_i - d_i)$ where f_i is completion time. Prove optimality via an exchange argument (adjacent swap).

Chapter Notes

Huffman (1952) and Kruskal (1956)/Prim (1957) are textbook examples of matroid optimisation; interval scheduling is a special case of weighted interval scheduling (Ch. 7, DP). The U.S. coin system is canonical—Pearson (2005) gives a polynomial-time test for canonicity. For the general theory of when greedy succeeds, see matroids and greedoids.

Chapter 6

Dynamic Programming

Dynamic programming (DP) solves optimisation problems by combining solutions to overlapping subproblems. Unlike divide-and-conquer, which solves independent subproblems, DP reuses work—trading space for time and often reducing exponential search to polynomial time.

6.1 The DP Paradigm

Two properties characterise problems amenable to DP.

6.1.1 Optimal Substructure

A problem has *optimal substructure* if an optimal solution contains optimal solutions to its subproblems. Formally, let S be an optimal solution to instance I and let I' be a sub-instance induced by S; then the restriction of S to I' is optimal for I'.

Not every optimisation problem qualifies. Longest *simple* path in a general graph lacks optimal substructure—the subpath of a longest simple path need not be longest between its endpoints, because reusing vertices may be forbidden globally but not locally. DP is inapplicable there.

6.1.2 Overlapping Subproblems

A recursive formulation has *overlapping subproblems* when the recursion tree revisits the same sub-instances many times. Naive recursion then does exponential work. DP stores each distinct subproblem result once.

Property	Divide & Conquer	Dynamic Programming
Subproblems	Independent / disjoint	Overlapping
Recomputation	None	Extensive without memo
Typical fix	–	Memoisation or tabulation

Table 6.1: DP versus divide-and-conquer.

6.1.3 Memoisation versus Tabulation

Both techniques achieve the same asymptotic bound; they differ in control flow.

- **Memoisation (top-down):** Write the natural recursion and cache results in a dictionary or array on first computation. Only subproblems reachable from the initial call are solved. Recursion overhead and stack depth remain.

- **Tabulation (bottom-up):** Enumerate subproblems in order of increasing size and fill a table iteratively. No recursion; often better cache locality and easier complexity analysis.

```
# memoised Fibonacci
memo = {0:0, 1:1}
def fib(n):
    if n not in memo:
        memo[n] = fib(n-1) + fib(n-2)
    return memo[n]

# tabulated Fibonacci
def fib_tab(n):
    dp = [0]*(n+1); dp[1]=1
    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

Tabulation is preferred in this course because it makes the dependency structure explicit, which we exploit to reconstruct solutions.

6.2 Longest Common Subsequence

Given $X = x_1 \dots x_m$ and $Y = y_1 \dots y_n$, a *subsequence* deletes zero or more characters. The LCS problem asks for a longest sequence that is a subsequence of both X and Y.

6.2.1 Optimal Substructure and Recurrence

Let $c[i][j]$ be the LCS length of $X_i = x_1 \dots x_i$ and $Y_j = y_1 \dots y_j$, with $c[0][j] = c[i][0] = 0$.

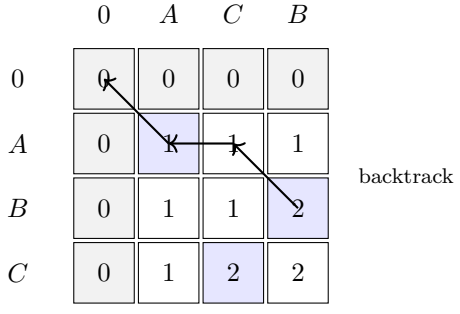
$$c[i][j] = \begin{cases} c[i-1][j-1] + 1 & \text{if } x_i = y_j, \\ \max\{c[i-1][j],\, c[i][j-1]\} & \text{if } x_i \neq y_j. \end{cases}$$

If $x_i = y_j$, any LCS must end with that character and extend an LCS of X_{i-1}, Y_{j-1}. If $x_i \neq y_j$, an LCS discards either x_i or y_j. Overlapping subproblems are immediate: $c[i][j]$ depends on three neighbours, so naive recursion re-evaluates the same cells exponentially many times.

6.2.2 Tabulation and Walkthrough

Filling row by row gives $O(mn)$ time and $O(mn)$ space. We also store a direction $b[i][j] \in \{\nwarrow, \uparrow, \leftarrow\}$ to reconstruct an LCS by backtracking from $c[m][n]$.

Take $X = \langle A, B, C \rangle$, $Y = \langle A, C, B \rangle$ ($m = n = 3$):



c table; blue cells = matches, arrows = backtrack for LCS $\langle A, B \rangle$

Filling order: $c[1][1] = 1$ (A=A, diagonal +1), $c[1][2] = 1$ ($\max(0, 1)$), $c[1][3] = 1$, $c[2][3] = 2$ (B=B). Backtracking from $c[3][3] = 2$ follows the arrows above to recover $\langle A, B\rangle$ (equivalently $\langle A, C\rangle$ via a different tie-break).

Reconstruction is $O(m + n)$: start at (m, n); on $\nwarrow$ output x_i and move to $(i - 1, j - 1)$; on $\uparrow$ move to $(i - 1, j)$; on $\leftarrow$ move to $(i, j - 1)$.

6.3 0/1 Knapsack

Given n items with weights w_i, values v_i, and capacity W, choose a subset maximising total value without exceeding W. Each item is taken at most once.

Let $K[i][w]$ be the optimum value using items $1 \dots i$ with capacity w, where $K[0][w] = K[i][0] = 0$:

$$K[i][w] = \begin{cases} K[i-1][w] & \text{if } w_i > w, \\ \max\{K[i-1][w],\ K[i-1][w - w_i] + v_i\} & \text{otherwise.} \end{cases}$$

Either item i is excluded (value $K[i-1][w]$) or included (value $K[i-1][w - w_i] + v_i$). Time $O(nW)$, space $O(nW)$—pseudo-polynomial in W.

6.3.1 Table and Backtracking

Example: $W = 5$, items (w, v): $1 : (2, 3), 2 : (3, 4), 3 : (4, 5), 4 : (5, 6)$.

$i \backslash w$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	3	4	7
3	0	0	3	3	4	7
4	0	0	3	3	4	7

backtrack: take 2,1

Optimum $K[4][5] = 7$ (items 1+2, weight 5). Backtrack: if $K[i][w] \neq K[i-1][w]$, item i was taken; move to $w - w_i$, else move to $i - 1$. Space can be reduced to $O(W)$ with a 1-D array iterated backwards, but backtracking then needs the full table or stored decisions.

6.4 Matrix Chain Multiplication

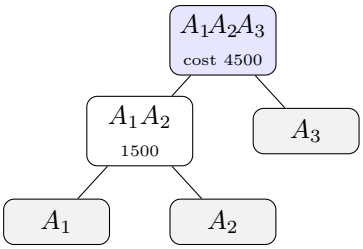
Given matrices $A_1 \dots A_n$ where A_i is $p_{i-1} \times p_i$, the cost of multiplying $A_i \cdots A_j$ depends on parenthesisation. Multiplying $p \times q$ by $q \times r$ costs pqr scalar multiplications.

Let $m[i][j]$ be the minimum cost to compute $A_i \cdots A_j$, with $m[i][i] = 0$:

$$m[i][j] = \min_{i \le k < j} \{m[i][k] + m[k+1][j] + p_{i-1}\, p_k\, p_j\}.$$

We also store $s[i][j] = k$ achieving the minimum to reconstruct the parenthesisation. There are $\Theta(n^2)$ subproblems, each trying $O(n)$ splits: $O(n^3)$ time, $O(n^2)$ space. Fill by increasing chain length $l = 2 \dots n$.

Example: $p = (10, 30, 5, 60)$ for $A_1(10\times30), A_2(30\times5), A_3(5\times60)$. Then $m[1][3] = \min\{m[1][1] + m[2][3] + 10{\cdot}30{\cdot}60 = 9000 + 3000,\ m[1][2] + m[3][3] + 10{\cdot}5{\cdot}60 = 1500 + 3000\} = 4500$ with $k = 2$, i.e. $(A_1 A_2) A_3$.



Optimal parenthesisation tree: $(A_1A_2)A_3$

6.5 Coin Change: Unbounded Knapsack

Given coin denominations $d_1 \dots d_k$ (unlimited supply) and amount N, find the minimum number of coins to make N (or report impossible).

Let $dp[x]$ be the minimum coins for amount x, with $dp[0] = 0$ and $dp[x] = \infty$ for $x < 0$ initially. For $x \geq 1$:

$$dp[x] = 1 + \min_{d_i \leq x} dp[x - d_i],$$

taking ∞ if no $d_i \leq x$ yields a finite predecessor. Equivalently, tabulate $x = 1 \dots N$ and for each coin d_i relax $dp[x]$ from $dp[x - d_i]$.

This is unbounded knapsack: each coin may be reused, so the recurrence looks back at the same coin set (contrast 0/1 knapsack which moves to $i - 1$). Time $O(kN)$, space $O(N)$. Reconstruct coins by storing $choice[x]$, the first coin used.

Example: $d = (1, 3, 4)$, $N = 6$. Then dp is $0, 1, 2, 1, 1, 2, 2$ for $x = 0 \dots 6$; e.g. $dp[6] = 1 + \min(dp[5], dp[3], dp[2]) = 1 + 1 = 2$ via $3 + 3$. Greedy (always take largest $d_i \leq x$) fails here: it gives $4 + 1 + 1$ (3 coins).

```
def coin_change(coins, N):
    INF = N+1
    dp = [INF]*(N+1); choice=[-1]*(N+1)
    dp[0]=0
    for x in range(1, N+1):
        for c in coins:
            if c<=x and dp[x-c]+1 < dp[x]:
                dp[x]=dp[x-c]+1; choice[x]=c
    return dp[N] if dp[N]!=INF else -1  # -1 = impossible
```

The same table answers counting variants: number of ways uses $ways[x] += ways[x - d_i]$ with careful ordering to avoid permuting combinations.

6.6 Exercises

1. **LCS by hand.** Compute the full c and b tables for $X = \langle B, D, C, A \rangle$, $Y = \langle A, B, C, D \rangle$. List all distinct LCSs and trace each backtracking path.

2. **Knapsack trace.** For capacity $W = 6$ and items $(w, v) = (1, 1), (2, 4), (3, 4), (4, 5)$, fill the K table, state the optimum value, and backtrack to find every optimal subset. Then give the $O(W)$-space tabulated code and explain why the inner loop must run backwards.

3. **Matrix chain.** Let $p = (5, 10, 3, 12, 5)$. Compute all $m[i][j]$ and $s[i][j]$ by hand, give the optimal parenthesisation and its cost, and argue why the greedy choice of cheapest single multiplication first is not optimal.

4. **Coin change variants.** With $d = (1, 3, 4)$ and $N = 6$, (a) show the dp table and the coins chosen, (b) give a counterexample where greedy fails for $d = (1, 7, 10)$, and (c) modify the DP to count the number of distinct combinations (order irrelevant) and compute it for $N = 6$.

5. **Edit distance (DP design).** Define edit distance with operations insert, delete, replace (cost 1). Derive the recurrence for $ed[i][j]$ on prefixes, state the table size and filling order, and adapt the algorithm to output an optimal edit sequence.

Chapter 7

Graph Traversal, Shortest Paths, and String Matching

> *"Graphs and strings are not different worlds."* BFS discovers distance as surely as KMP discovers pattern: both exploit structure to avoid redundant work.

Prerequisites: Chapter 1 (invariants), Chapter 2 (asymptotics, heaps), and basic graph notation $G = (V, E)$ directed or undirected, $|V| = n$, $|E| = m$.

Learning Outcomes

After this chapter you will be able to:

(L1) trace BFS/DFS, explain queue/stack roles and analyse $O(n + m)$ time;

(L2) implement Dijkstra with a binary heap and prove correctness via its set invariant;

(L3) run Bellman–Ford, detect negative cycles, and derive Floyd–Warshall as DP over k;

(L4) compute the KMP prefix function π, simulate its automaton, and analyse Rabin–Karp rolling hash.

7.1 Graph Traversal: BFS and DFS

Let $G = (V, E)$ be unweighted for traversal; weights return in §7.2. We assume adjacency-list representation; adjacency matrix would cost $\Theta(n^2)$ per scan.

7.1.1 BFS: Queue and Layering

BFS(G, s) explores in waves of increasing distance from source s. It uses a FIFO *queue* Q: s is enqueued, then repeatedly dequeued, and each undiscovered neighbour is enqueued.

```
BFS(G, s)
1 dist[s] ← 0; parent[s] ← NIL; Q ← [s]; visited← {s}
2 while Q ≠ ∅ do
3   u ← DEQUEUE(Q)
4   for each (u, v) ∈ E do
5     if v ∉ visited then visited∪{v}; dist[v] ←dist[u]+1; parent[v] ← u; ENQUEUE(Q, v)
```

The *layering* property: Q always contains vertices of at most two consecutive distances $d, d + 1$ from s, all d before $d + 1$. Hence dist$[v]$ is the shortest-path distance $\delta(s, v)$ in an unweighted graph (proved by induction on d). Parent pointers form a *BFS tree* rooted at s; every non-tree edge connects vertices whose layers differ by at most one.

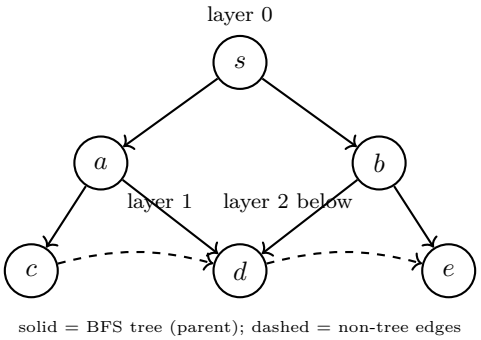


Figure 7.1: BFS layering from s. Queue order visits layer 0, then layer 1 (a, b), then layer 2 (c, d, e). Each vertex is enqueued once; every edge examined once (twice if undirected): $O(n+m)$ time and $O(n)$ extra space.

Example 7.1 (BFS distances on Fig. 7.1)**.** Queue evolution: $[s] \to [a, b] \to [b, c, d] \to [c, d, e] \to [d, e] \to [e] \to []$. Thus $\text{dist}[s] = 0$, $\text{dist}[a] = \text{dist}[b] = 1$, $\text{dist}[c] = \text{dist}[d] = \text{dist}[e] = 2$; parent pointers give shortest-path tree $s \to a \to c$, $s \to \{a, b\} \to d$, $s \to b \to e$. Any $s \to e$ path needs at least 2 edges, and BFS finds one.

Analysis: each vertex enters Q once and each adjacency list is scanned once, giving $O(n+m)$ time if G is stored as adjacency lists. BFS also tests bipartiteness (2-colour by layer parity; an edge within a layer $\Rightarrow$ odd cycle) and finds connected components by restarting from any unvisited vertex. Multi-source BFS (enqueue all sources at distance 0) solves "nearest source" or "shortest in unweighted grid" problems in the same bound.

7.1.2 DFS: Stack and Structure

DFS replaces the queue by a LIFO *stack* (or recursion): go deep before going wide. Iterative version pushes s onto a stack; at each step pop u and push its undiscovered neighbours.

```
DFS-Visit(u): mark u discovered; for each (u,v) ∈ E if v undiscovered then parent[v] ←
u; DFS-Visit(v); mark u finished.
```

With discovery/finish times $d[u], f[u]$, DFS classifies edges (undirected: tree/back; directed: tree/back/forward/cross) and yields the *parenthesis theorem*: intervals $[d[u], f[u]]$ are nested or disjoint. This structure powers two classic applications: cycle detection (back edge $\Leftrightarrow$ cycle in directed graph) and topological sort (order vertices by decreasing $f[u]$ for a DAG). Kosaraju's two-pass DFS finds strongly connected components. Like BFS, DFS scans each vertex and edge once: $O(n + m)$ time, $O(n)$ stack/colour array.

Queue vs. stack intuition: BFS's queue enforces breadth (fairness across branches); DFS's stack enforces depth (commitment to one branch). Swapping the container is the only algorithmic difference, yet the resulting visit orders and edge classifications diverge sharply.

7.2 Shortest Paths

Edge weights $w : E \to \mathbb{R}$. Let $\delta(s, v)$ denote the true shortest-path weight ($-\infty$ if a negative cycle is reachable from s on some $s \to v$ path, ∞ if v is unreachable). All algorithms below use $\textsc{Relax}(u, v)$: if $\text{dist}[v] > \text{dist}[u] + w(u, v)$ then update $\text{dist}[v]$ and $\text{parent}[v] \leftarrow u$.

7.2.1 Dijkstra: Greedy with a Heap

Assumption: $w(e) \geq 0$ for all e. Dijkstra grows a set S of settled vertices whose final distances are known, repeatedly extracting the unsettled vertex with minimum tentative distance.

```
DIJKSTRA(G, w, s)
1 dist[s] ← 0; dist[v ≠ s] ← ∞; S ← ∅; min-heap Q on dist
2 while Q ≠ ∅ do
3   u ← EXTRACTMIN(Q); S ← S ∪ {u}
4   for each (u, v) ∈ E do RELAX(u, v): if dist[v] >dist[u]+w(u, v) then dist[v] ←dist[u]+
w(u, v); DECREASEKEY(Q, v)
```

Heap implementation: binary heap gives $O((n+m) \log n)$; Fibonacci heap improves to $O(m+ n \log n)$. Early exit when a target t is extracted — its distance is already final. Relaxation is the only place dist decreases, mirroring the BFS layering but weighted.

Correctness — invariant: At the start of each iteration with S already settled,

$$\text{(I)} \quad \forall\, x \in S :\ \text{dist}[x] = \delta(s, x) \quad \text{and} \quad \forall\, y \notin S :\ \text{dist}[y] = \min_{P:\, s \to y \text{ with inner vertices in } S} w(P).$$

Initialisation: $S = \emptyset$, (I) holds vacuously ($\text{dist}[s] = 0$). *Maintenance:* Let $u \notin S$ have minimum dist among $V \setminus S$. Any $s \to u$ path must leave S at some edge (x, y) with $x \in S$, $y \notin S$; its weight is $\geq \text{dist}[y] \geq \text{dist}[u]$ because all weights are non-negative, so no unseen path beats $\text{dist}[u]$. Extracting u preserves (I) after relaxing its edges. *Termination:* $S = V$ gives $\text{dist}[v] = \delta(s, v)$ for all v. If w has a negative edge the inequality fails and Dijkstra may err — a standard counterexample is $s \xrightarrow{2} b \xrightarrow{-2} a$ where a is settled too early.

7.2.2 Bellman–Ford: Relaxation and Negative Cycles

No non-negativity needed. Relax every edge $n - 1$ times, in any order:

```
for i ← 1 to n − 1 do for each (u, v) ∈ E do RELAX(u, v)
```

After k rounds, $\text{dist}[v]$ equals the shortest weight among $s \to v$ paths using $\leq k$ edges (induction on k: the k-edge optimum extends a $(k-1)$-edge optimum by one relaxation). Since any shortest simple path uses at most $n - 1$ edges, after $n - 1$ rounds dist$= \delta$ iff no negative cycle is reachable. One extra round detects one: if any relaxation succeeds, a negative cycle exists and $\delta = -\infty$ on its reachable vertices; tracing parent pointers retrieves it.

Example 7.2 (Bellman–Ford detects a negative cycle)**.** Vertices $\{s, a, b\}$ with $s \xrightarrow{1} a$, $a \xrightarrow{1} b$, $b \xrightarrow{-3} a$. After $n - 1 = 2$ rounds $\text{dist}[a] = 1$, $\text{dist}[b] = 2$. Round 3 relaxes $b \to a$: $\text{dist}[a] > 2 + (-3) = -1$, so $\text{dist}[a]$ drops — negative cycle $a \to b \to a$ of weight -2 detected.

Time $O(nm)$, space $O(n)$ — slower than Dijkstra but fully general.

7.2.3 Floyd–Warshall: DP over Intermediate Vertices

For all-pairs shortest paths, let $d^{(k)}[i,j]$ be the shortest $i \to j$ weight whose *inner* vertices lie in $\{1,\dots,k\}$ (endpoints unrestricted). Then

$$d^{(0)}[i,j] = \begin{cases} 0 & i = j \\ w(i,j) & (i,j) \in E \\ \infty & \text{else} \end{cases}, \qquad d^{(k)}[i,j] = \min\bigl(d^{(k-1)}[i,j],\ d^{(k-1)}[i,k] + d^{(k-1)}[k,j]\bigr).$$

Iterating $k = 1 \dots n$ in place (a single $n \times n$ matrix suffices since $d^{(k)}[i,k] = d^{(k-1)}[i,k]$ and $d^{(k)}[k,j] = d^{(k-1)}[k,j]$) yields $\delta(i,j) = d^{(n)}[i,j]$. Time $\Theta(n^3)$, space $\Theta(n^2)$. Negative-cycle check: $\exists\, i :\ d^{(n)}[i,i] < 0$. Despite cubic cost, Floyd is concise, cache-friendly, and reconstructs paths via a successor matrix $\mathrm{nxt}[i,j]$ updated alongside d.

Remark 7.1 (Choosing among shortest-path algorithms). Non-negative weights and single source: Dijkstra. Negative weights or cycle detection needed: Bellman–Ford. Dense all-pairs or transitive closure: Floyd–Warshall. On sparse graphs, running Dijkstra from each source with a heap can beat Floyd at $O(n(m \log n))$.

7.3 String Matching

Text $T[1 \dots n]$, pattern $P[1 \dots m]$, alphabet Σ ($n \geq m$). Report all shifts s with $T[s+1 \dots s+m] = P$. We compare three strategies that trade comparison cost for preprocessing.

7.3.1 Naïve $\Theta((n-m+1)m)$ Scan

Slide P over each alignment $s = 0 \dots n-m$ and compare $P[1 \dots m]$ against $T[s+1 \dots s+m]$ character by character, breaking on first mismatch. Worst case $\Theta((n-m+1)m)$ — e.g., $T = a^n$, $P = a^{m-1}b$ forces m comparisons at each of $n-m+1$ positions. No preprocessing, $O(1)$ extra space. The next two algorithms avoid re-scanning characters already matched.

7.3.2 KMP: Prefix Function and Automaton

Let $\pi[q] = \max\{k < q : P[1 \dots k] \text{ is suffix of } P[1 \dots q]\}$ for $1 \leq q \leq m$ ($\pi[1] = 0$); equivalently $\pi[q]$ is the longest proper border of the prefix $P[1 \dots q]$. Computed in $O(m)$ by amortised fallback:

$\pi[1] \leftarrow 0;\ k \leftarrow 0$; **for** $q \leftarrow 2$ **to** m **do while** $k > 0 \wedge P[q] \neq P[k+1]$ **do** $k \leftarrow \pi[k]$; **if** $P[q] = P[k+1]$ **then** $k \leftarrow k+1;\ \pi[q] \leftarrow k$

Matching scans T once, maintaining q = length of longest prefix of P that is a suffix of $T[1 \dots i]$; on mismatch fall back $q \leftarrow \pi[q]$. Total $O(n+m)$ time, $O(m)$ extra space; each character of T is examined at most twice.

The automaton view: state q means "first q characters of P just matched." Transition $\delta(q,c)$ is the longest prefix of P that is a suffix of $P[1 \dots q]c$; missing edges follow π-links (failure function).

7.3.3 Rabin–Karp: Rolling Hash

Treat strings as base-b numbers modulo a prime q. Let $h(s) = T[s+1 \dots s+m] \bmod q$ and $p = P \bmod q$. With precomputed $b^{m-1} \bmod q$,

$$h(s+1) = \bigl(b \cdot (h(s) - T[s+1] \cdot b^{m-1}) + T[s+m+1]\bigr) \bmod q,$$

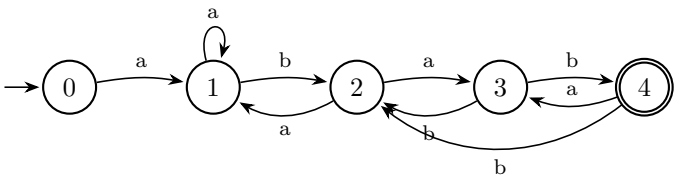


Figure 7.2: KMP automaton for $P = \texttt{abab}$ ($m = 4$). Solid spine spells P; failure links (merged into δ) redirect on mismatch via π: $\pi = [0, 0, 1, 2]$. Scanning T never moves backwards, giving $O(n + m)$ time.

so each shift is $O(1)$ after $O(m)$ preprocessing. If $h(s) \neq p$ there is no match; if equal, verify by direct comparison (handles spurious hits from collisions). Expected $O(n + m)$ with good q and $O(nm)$ worst case when every window collides.

Example 7.3 (Rolling hash by hand)**.** Let $b = 10$, $q = 13$ (small for illustration), $T = \texttt{31415}$, $m = 2$. Then $h(0) = \texttt{31} \bmod 13 = 5$, $h(1) = (10 \cdot (5 - 3 \cdot 10) + \texttt{4}) \bmod 13 = 1$ which equals $\texttt{14} \bmod 13 = 1$; $p = \texttt{15} \bmod 13 = 2$, so neither window matches $P = \texttt{15}$. With real $b = 256$, $q \approx 10^9 + 7$ and Horner evaluation, the same recurrence applies to strings over bytes.

Double hashing or a large $q \gg n$ makes collisions negligible in expectation. Rabin–Karp extends naturally to 2-D pattern search and multi-pattern search with one text scan. In practice it shines when many patterns are sought in one long text, e.g., plagiarism detection over a corpus.

7.4 Chapter Notes

BFS (Moore, 1959) and DFS (Tarjan, 1972) in modern form underpin all graph algorithms. Dijkstra (1959) with binary heaps is the practical default; Bellman–Ford (1958) and Floyd–Warshall (1962) handle negative weights. KMP (Knuth–Morris–Pratt, 1977) and Rabin–Karp (1987) remain textbook representatives of automaton vs. hashing. CLRS Chs. 20–24, 32 covers all topics in depth.

7.5 Exercises

E7.1 **BFS layering vs. DFS depth.** Run BFS and DFS (alphabetical adjacency) from s on the graph of Fig. 7.1 with extra edge $e \to a$. List discovery order, dist[·] from BFS, and DFS finish times $f[\cdot]$. Give a graph where DFS from s never discovers a vertex that BFS does. Can that happen if the graph is undirected and connected? Justify.

E7.2 **Dijkstra proof in action.** (a) Execute Dijkstra with a binary heap from s on: $s \xrightarrow{4} a$, $s \xrightarrow{2} b$, $b \xrightarrow{1} a$, $a \xrightarrow{3} t$, $b \xrightarrow{5} t$. Show heap contents per extraction and invariant (I). (b) Add edge $b \xrightarrow{-3} a$ and show extraction order violates (I) and yields a wrong distance. (c) Would Bellman–Ford recover the correct distances and how many rounds suffice?

E7.3 **KMP prefix function.** (a) Compute π for $P = \texttt{aabaaab}$ step by step. (b) Draw the KMP automaton states $0 \ldots 7$ and label failure (π) links. (c) Trace KMP on $T = \texttt{aabaabaaab}$ and count character comparisons versus naïve scan. Prove $O(n + m)$ is tight by giving a family attaining $\Theta(n + m)$ comparisons.

E7.4 **Rolling hash design.** Let $b = 256$, $q = 101$. (a) Compute the Rabin–Karp hash of $P = \texttt{abc}$ and the rolling update to the next window in $T = \texttt{abcd}$. (b) Exhibit $P \neq Q$ over $\{a, b\}$ colliding modulo 101, and explain why a second modulus q_2 reduces collision probability

to $\approx 1/(q\,q_2)$. (c) Argue expected time is $O(n+m)$ when $q \geq m^2$ and verification is $O(m)$, assuming uniform hashing.

Chapter 8

NP-Completeness

> *"If P =NP, the world would change profoundly; if P ≠NP, we must cope."* NP-completeness tells us which problems admit efficient algorithms and which almost certainly do not.

Prerequisites: Asymptotic analysis, polynomial time, reductions, and graph basics (Vertex Cover, Clique).

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish P, NP, coNP via verifiers and certificates;

(L2) define polynomial reduction and NP-completeness (Cook–Levin) and chain reductions SAT → 3SAT → Clique → Vertex Cover;

(L3) prove the 2-approximation for Vertex Cover and bound greedy Set Cover;

(L4) discuss P vs. NP and select coping strategies (approximation, heuristics) for NP-complete problems.

8.1 P, NP, and coNP

A *language* $L \subseteq \{0,1\}^*$ is a decision problem: $x \in L$ means "yes". We measure time in the input length $n = |x|$ on a deterministic RAM/Turing model.

Definition 8.1 (P, informal)**.** P is the class of languages decidable by a deterministic algorithm in polynomial time, i.e. $O(n^c)$ for some constant c.

Think: sorting, BFS, Dijkstra, and linear programming are in P—routine work even for large n when c is small.

Definition 8.2 (NP, verifier view)**.** $L \in$ NP if there exists a deterministic polynomial-time *verifier* $V(x, c)$ and polynomial p such that $x \in L \iff \exists\, c$ with $|c| \le p(|x|)$ and $V(x, c) = 1$. The string c is the *certificate* (or witness): it proves membership and can be checked quickly, even if finding it is hard.

Equivalently, L is decided by a nondeterministic machine that guesses c then verifies. Example: 3SAT certificate is a satisfying assignment; Clique certificate is the k vertices. Clearly P ⊆ NP: ignore the certificate and decide directly.

Remark 8.1 (coNP). coNP = $\{\overline{L} : L \in \mathsf{NP}\}$: languages whose "no"-answers have short certificates. Example: UNSAT (formula has no satisfying assignment) is in coNP but not known to be in NP. We have $\mathsf{P} \subseteq \mathsf{NP} \cap \mathsf{coNP}$; whether $\mathsf{NP} = \mathsf{coNP}$ is open and would imply short proofs of unsatisfiability.

Example 8.1 (Certificates at work)**.** For Clique(G, k), the certificate lists k vertices; the verifier checks $\binom{k}{2}$ edges in $O(k^2) \leq O(n^2)$ time. For Subset Sum(S, t), the certificate is the subset itself. In both cases verification is easy while search appears hard—the essence of NP.

A useful intuition: P = can *find* quickly; NP = can *check* quickly given a hint. Whether hints ever help asymptotically is exactly P vs. NP.

8.2 Polynomial Reductions and NP-Completeness

Definition 8.3 (Polynomial reduction)**.** $A \leq_p B$ (A reduces to B) if there is a polynomial-time computable f with $x \in A \iff f(x) \in B$ for all x. Intuition: B is at least as hard as A; a solver for B solves A with one call plus the cost of f.

Definition 8.4 (NP-complete, NP-hard)**.** L is NP-hard if every $A \in \mathsf{NP}$ satisfies $A \leq_p L$. L is NP-complete if $L \in \mathsf{NP}$ and L is NP-hard. Thus an NP-complete problem is a hardest problem in NP: a polynomial algorithm for one yields one for all.

Theorem 8.1 (Cook–Levin, stated without proof)**.** SAT is NP-complete.

Every NP verification can be encoded as a Boolean circuit/tableau whose satisfiability mirrors the existence of a certificate. From this single seed, hundreds of problems inherit completeness by reduction chains. Since $\leq_p$ is transitive (compose the polynomial maps), it suffices to reduce from any known NP-complete problem rather than from all of NP.

8.2.1 SAT $\leq_p$ 3SAT: clause splitting

A SAT clause $(l_1 \vee \cdots \vee l_k)$ with $k > 3$ is split using fresh variables y_i:

$$(l_1 \vee l_2 \vee y_1) \wedge (\bar{y}_1 \vee l_3 \vee y_2) \wedge \cdots \wedge (\bar{y}_{k-3} \vee l_{k-1} \vee l_k).$$

Clauses with 1 or 2 literals are padded by repeating a literal. The new 3-CNF is satisfiable iff the original is, and $|f(x)| = O(|x|)$. Hence SAT$\leq_p$3SAT and 3SAT is NP-complete.

Example 8.2 (Clause splitting)**.** $(x_1 \vee \bar{x}_2 \vee x_3 \vee x_4)$ becomes $(x_1 \vee \bar{x}_2 \vee y_1) \wedge (\bar{y}_1 \vee x_3 \vee y_2) \wedge (\bar{y}_2 \vee x_4 \vee x_4)$. Any assignment satisfying the original extends to the y_i (set $y_1 = 0$ if $x_1 \vee \bar{x}_2$ true, else 1); conversely, any satisfying assignment for the 3-CNF restricts to one for the original.

8.2.2 3SAT $\leq_p$ Clique: triangles and conflicts

Given 3-CNF $\phi = C_1 \wedge \cdots \wedge C_m$ where $C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3})$, build G with $3m$ vertices—a triangle per clause (one vertex per literal). Add edges between all pairs except (i) within-clause non-edges are kept as triangle edges and (ii) *conflict edges* are omitted between x and $\bar{x}$ across clauses. Set $k = m$. Then G has a k-clique iff ϕ is satisfiable: pick one true literal per clause; they are pairwise non-conflicting. Size is $O(m^2)$, polynomial.

8.2.3 Clique $\leq_p$ Vertex Cover: complement

For $G = (V, E)$, $\bar{G} = (V, \bar{E})$ is the complement. $S \subseteq V$ is a clique in G iff $V \setminus S$ is a vertex cover in $\bar{G}$: an edge missing from S in G becomes an uncovered edge in $\bar{G}$ and vice versa. So $(G, k) \mapsto (\bar{G}, |V| - k)$ is a reduction, and Vertex Cover is NP-complete. Chaining further, Vertex Cover$\leq_p$Set Cover, Hamiltonian Cycle, and Subset Sum are also NP-complete; any NP-hard problem not known to be in NP (e.g. halting) is called NP-hard only.

Example 8.3 (Clique vs. Vertex Cover)**.** In $G = C_4$ (4-cycle), $\{1, 3\}$ is not a clique but $\{2, 4\} = V \setminus \{1, 3\}$ covers all edges of $\bar{G}$ (which is $2K_2$ plus diagonals). Complement swaps "all pairwise adjacent" with "every edge touched."

Remark 8.2 (Hard vs. complete). NP-complete $\subseteq$ NP; NP-hard may lie outside. Halting and optimisation versions ("find min vertex cover") are NP-hard but not decision problems in NP. Showing $L \in$ NP is half the completeness proof.

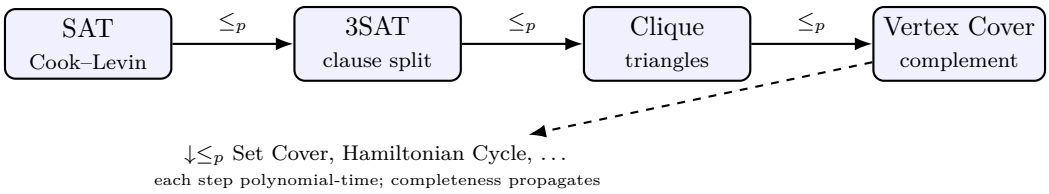


Figure 8.1: Reduction chain from Cook–Levin. Hardness flows rightward: solving any downstream problem in polynomial time would collapse the whole chain.

8.3 Coping with NP-Completeness

When a problem is NP-complete we do not expect an exact polynomial algorithm. SC2001 emphasises three coping strategies: bounded approximation, logarithmic approximation, and fast heuristics without guarantees.

8.3.1 2-approximation for Vertex Cover via maximal matching

Approx-VC: pick any maximal matching M (greedily add disjoint edges until stuck) and return $C =$ all endpoints of M.

Any maximal matching is a lower bound on OPT: disjoint edges force distinct cover vertices. This yields a simple 2-approximation.

Theorem 8.2. C is a vertex cover with $|C| \leq 2 \cdot \mathrm{OPT}$.

Proof. M is maximal, so every edge shares an endpoint with some $e \in M$; thus C covers all edges. Any vertex cover must pick at least one endpoint per edge of M, and edges of M are disjoint, so $\mathrm{OPT} \geq |M|$. But $|C| = 2|M| \leq 2 \cdot \mathrm{OPT}$. Tight example: a complete bipartite $K_{n,n}$ has $\mathrm{OPT} = n$, $M = n$, $|C| = 2n$. Time $O(n + m)$: scan edges once to build M, then output C. No need to find a maximum matching—any maximal one suffices, which is why the algorithm is linear.

8.3.2 Greedy Set Cover: $\ln n$ approximation

Repeatedly pick the set covering the most uncovered elements. If $\mathrm{OPT} = k$ and n elements remain, some set covers $\geq n/k$ of them, so n shrinks geometrically. Standard charging gives $|\textsc{Greedy}| \leq H_n \cdot \mathrm{OPT} = O(\log n) \cdot \mathrm{OPT}$ where $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$, and this is essentially optimal unless P = NP (Feige's $\Omega(\log n)$ inapproximability). The same greedy idea underlies many log-factor approximations.

8.3.3 Heuristic for TSP: nearest neighbour

Start at any city, repeatedly go to the nearest unvisited city, return to start. No constant-factor guarantee exists for general TSP (unless P = NP); on metric instances it can be $\Theta(\log n)$ off

and fails badly on crafted examples, but it is $O(n^2)$ and often reasonable in practice. Better guarantees need Christofides (1.5-approx on metric TSP) or local search/metaheuristics.

Beyond approximation, practical coping includes: (i) *parameterised algorithms*—exponential only in a small parameter k (e.g. Vertex Cover in $O(2^k \cdot (n+m))$ via bounded search); (ii) *exact exponential* methods with pruning (branch-and-bound, SAT solvers that routinely handle thousands of variables); (iii) *randomisation and local search*—simulated annealing, genetic algorithms, and large-neighbourhood search for industrial TSP/VRP. SC2001 LO focuses on (i)–(iii) above via greedy bounds, but recognising the broader toolbox helps choose the right trade-off for the instance at hand.

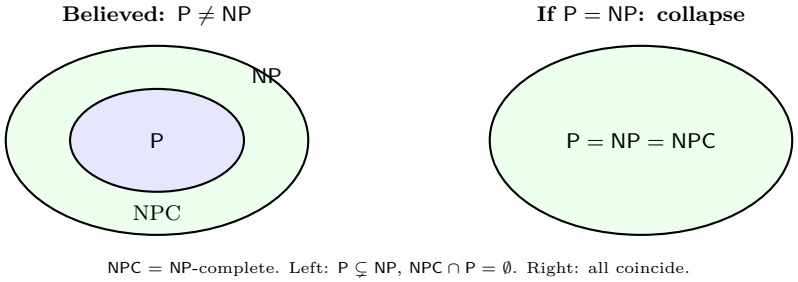


Figure 8.2: Two views of P vs. NP. The left is widely believed; the right would make every NP problem tractable.

8.4 P vs. NP and Why P ≠ NP Is Believed

P vs. NP asks whether every efficiently verifiable problem is efficiently solvable. A proof either way would be momentous: P = NP would give polynomial algorithms for all NP-complete problems (cryptography breaks, optimisation becomes easy); P ≠ NP confirms inherent hardness. Why should we believe P ≠ NP? Three strands reinforce each other. (i) decades of failed search for polynomial algorithms despite intense effort, (ii) oracles/barriers suggesting techniques relativise or algebrize insufficiently, and (iii) cryptographic practice whose security would otherwise collapse. A polynomial algorithm for any NP-complete problem would propagate via $\leq_p$ to all of NP (Fig. 8.1). Concretely, if P = NP then integer factoring, Nash equilibria, and protein folding would all admit efficient algorithms—an extraordinary consequence with no evidence. Yet belief is not proof—a valid separation must rule out *all* polynomial algorithms, a far stronger task than exhibiting one.

Reduction size check. Each construction above is polynomial: SAT → 3SAT blows each k-clause into $k-2$ clauses of size 3 with $k-3$ fresh variables; 3SAT → Clique creates $3m$ vertices and $O(m^2)$ edges; Clique → Vertex Cover complements the graph in $O(n^2)$. No exponential blowup hides in the gadget.

8.5 Chapter Notes

Cook (1971) and Levin (1973) founded NP-completeness; Karp (1972) listed 21 reductions. Garey–Johnson remains the catalogue. For approximation, Vazirani and Williamson–Shmoys cover Vertex Cover and Set Cover; TSP heuristics are surveyed in Applegate et al. A final perspective: NP-completeness is not a dead end but a signpost. It tells the algorithm designer to stop hunting for a worst-case polynomial exact algorithm and instead negotiate the specification—approximate, parameterise, or exploit instance structure. CLRS Ch. 34 is the standard concise reference. Sipser Ch. 7 gives the language-theoretic view of P/ NP/ coNP; Arora–Barak Ch. 2

covers Cook–Levin in full. Exam tip: to prove a new problem X is NP-complete, show (a) $X \in \mathsf{NP}$ (exhibit verifier) and (b) $Y \leq_p X$ for some known NP-complete Y; transitivity does the rest. For hands-on reduction practice, start from 3SAT or Vertex Cover—both are the most reusable sources in exams and interviews.

8.6 Exercises

Exercise 8.1 (Certificate vs. decision)**.** Give explicit verifiers (and certificate size bounds) for Subset Sum and Hamiltonian Cycle to show each is in NP. Explain why the complement of each is not obviously in NP and what that says about coNP.

Exercise 8.2 (Reductions by hand)**.** (a) Convert $\phi = (x_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee x_3) \wedge (x_1 \vee \bar{x}_3 \vee x_4 \vee \bar{x}_5)$ to an equisatisfiable 3-CNF via clause splitting; count new variables/clauses. (b) Apply the 3SAT→Clique construction to your 3-CNF and draw G; identify the m-clique for assignment $x_1 = 1, x_2 = 0, x_3 = 1$. (c) Take that G and exhibit the complementary vertex cover.

Exercise 8.3 (Approximation analysis)**.** (a) Prove the 2-approximation bound for Approx-VC is tight by giving an infinite family where $|C| = 2 \cdot \mathrm{OPT}$. (b) Show greedy Set Cover achieves H_n by charging each newly covered element $1/k$ where k is the number newly covered. (c) Give a 4-city TSP instance where nearest-neighbour (from A) returns a tour $> 2\times$ optimal, and explain why no constant-factor guarantee is possible for general TSP unless $\mathsf{P} = \mathsf{NP}$.

Exercise 8.4 (P vs. NP consequences)**.** Assume $\mathsf{P} = \mathsf{NP}$. (a) Show every NP-complete language is then in P and outline how to solve 3SAT in polynomial time. (b) Explain why public-key cryptography based on factoring (in $\mathsf{NP} \cap \mathsf{coNP}$) would be at risk even though factoring is not known to be NP-complete. (c) If someone proves $\mathsf{NP} \neq \mathsf{coNP}$, what does that imply for P vs. NP?